

INTRODUZIONE AL DATA MINING

Santo Lo Bianco

1000061604

INTRODUZIONE

Il termine “**data mining**” si riferisce alla scoperta di modelli interessanti e possibilmente inaspettati in un insieme di dati.

Tali informazioni estratte dai dati, potrebbero portare all’inferenza di una nuova conoscenza, come supporto per nuove decisioni o come base di nuove operazioni. Ad esempio, si utilizzano gli algoritmi di machine learning per capire cosa vogliono gli utenti sulla base delle operazioni fatte precedentemente.

L’obiettivo del data mining è trovare una nuova conoscenza e il machine learning potrebbe essere uno dei metodi per raggiungere questa conoscenza.

All’attività del data mining, si accompagnano due operazioni ugualmente importanti:

- **Data-cleaning:** rimuovere dati fittizi e artefatti, in modo che si possa lavorare su dati puliti;
 - Estrazione delle feature all’interno di un dataset grezzo;
 - Pulizia dei dati, togliendo entries erronee e fare una stima dei dati mancanti;
 - Selezione delle feature più rilevanti e trasformazione del nuovo set di dati, in uno adatto per l’analisi;
- **Visualizzazione dei dati:** mostrare i risultati che si sono ottenuti attraverso il data mining.

Le caratteristiche che devono avere i pattern estratti nel data mining sono:

- **Validi** -> devono essere verificabili e veri anche su dati nuovi; quindi, devono essere generalizzabili;
- **Utili** -> si conservano solo informazioni utili, tali da consentire operazioni e prendere decisioni;
- **Inattesi** -> devono portare alla conoscenza di informazioni, non facilmente derivabili;
- **Comprensibili** -> le informazioni riportate devono essere facilmente leggibili all’uomo.

Gli algoritmi di data mining possono essere di due tipi:

- **Descrittivi:** consentono di descrivere pattern comprensibili dall’uomo. Non si predice alcuna variabile, ma bensì si descrivono in modo dettagliato i dati. Ad esempio, nel clustering si raggruppano i dati omogenei e se ne descrivono le caratteristiche;
- **Predittivi:** a partire da dati noti, si descrivono possibili dati futuri. Si basa su modelli addestrati su dati storici, ad esempio la classificazione di email spam/no spam.

In un mondo dove si producono miliardi di informazioni al minuto se non al secondo, è utile applicare il data mining, perché avere tanti dati grezzi non significa avere conoscenza.

Ritrovarci ad applicare il data mining ad enormi quantità di dati, potrebbe portare a trovare conoscenze superflue, quindi pattern insignificanti.

Una caratteristica potrebbe risultare insignificante, nel momento in cui questa ha alta probabilità di essere osservata.

Ad esempio, durante il periodo del Covid, se si fossero osservate tutte le persone che avevano la febbre e, sulla base di questo dato, si fosse inferito che fossero state vittime di contagio, si sarebbe potuto incorrere in un’analisi fuorviante.

Infatti, la febbre poteva dipendere da molti altri fattori, e associare questa caratteristica a una causa specifica avrebbe potuto risultare erroneo.

Principio di Bonferroni

Supponendo di avere dati casuali, se il numero di eventi che soddisfano una certa caratteristica è maggiore o uguale al numero di volte in cui la stessa caratteristica si osserva nei dati reali, allora questa caratteristica non è un'evidenza, ma un artefatto dovuto al caso.

Si può parlare di pattern significativo solo quando il numero di casi osservati nei dati reali è molto maggiore rispetto a quanto ci si aspetterebbe in dati casuali.

PRE-PROCESSING DEI DATI

Prima di essere analizzati i dati potrebbero contenere alcune incongruenze quali:

- Provenienza da sorgenti diverse, bisogna **armonizzarli**-> uniformare formati, unità di misura ecc.;
- Contenere dati superflui, non rilevanti o rumorose, che portano ad un'analisi erronea-> si selezionano feature diverse o semplificando i dati mantenendo le informazioni più importanti;
- Errori o dati mancanti-> si eliminano o si completano con valori medi ecc. .

La fase di pre-processing dei dati inizia dopo la raccolta dei dati e si divide in vari steps:

- **Estrazioni dei giusti feature:** l'estrazione delle feature giuste può richiedere la comprensione dello specifico dominio applicativo in questione (l'ambito concreto in cui si applica il data mining).
- **Portabilità dei dati:** se un algoritmo lavora con un tipo di dato, allora è necessario convertire i dati per lo specifico algoritmo. Ad esempio, passare da dati numeri a categoriali, tipo nel caso dell'età che si deve inquadrare in un intervallo: $[0,10]$, $[10,20]$... $[90,100]$.
- **Cleaning dei dati.**
- **Riduzione dei dati:** ridurre la dimensionalità dei dati mediante tecniche di riduzione delle feature o trasformazione dei dati.

Estrazione dei giusti feature

Consiste nel trovare feature adatte a risolvere un problema. Ad esempio, per trovare e-mail spam potremmo verificare se: ci sono link collegati, se è spesso presente la parola "gratis", la lunghezza del testo è corrispondente ad altre viste in precedenza.

Portabilità dei dati

I dati potrebbero essere eterogenei e di tipo diverso, e alcuni algoritmi potrebbero supportare solo alcuni tipi di dati. Per questo si applicano conversioni, che alcune volte potrebbero risultare lossy.

La conversione da tipi numerici a categoriali è detta **discretizzazione**, la quale divide valori numerici in intervalli. $N \rightarrow [x,y]$

La discretizzazione può essere di tipo diverso:

- **Equi-width ranges** -> si divide il dominio dei valori in intervalli con le ampiezze equivalenti.
 - Esempio: abbiamo l'età il cui valore oscilla da 0 a 100 e la vogliamo dividere in 5 intervalli, allora faremo $100-0/5 = 20$. Tutti gli intervalli hanno larghezza 20.
- **Equi-log ranges** -> si divide il dominio dei valori in intervalli, dove ogni intervallo $[a,b]$ è costruito in modo che $\log_x(b)-\log_x(a)$ abbia per tutti lo stesso valore. Serve per dati che crescono in modo esponenziale o moltiplicativo.
 - Esempio: vogliamo suddividere le aziende per utile fatto in un anno e creiamo 4 intervalli:
 - 1 classe con utile che va da 1 a 100;

- 1 classe con utile che va da 100 a 1000;
- 1 classe con utile che va da 1000 a 10000;
- 1 classe con utile che va da 10000 a 100000;
- **Equi-depth ranges**-> si dividono i valori all'interno dell'intervallo in modo equo, in modo che ogni intervallo contenga lo stesso numero di record.
 - Esempio: vogliamo suddividere 100 studenti in modo che ci siano 4 classi:
 - 1 classe con studenti con la media più bassa(primi 25);
 - 1 classe con studenti con media superiore a 22(secondi 25);
 - 1 classe con studenti con la media più alta(terzi 25).
 - 1 classe con restanti studenti(quarti 25);

Si possono trasformare dei dati categoriali in dati numerici, associando ad ogni categoria ad un numero. Assegnando un numero potremmo pure ordinarli, ma in contesti dove l'ordine non deve esistere allora si potrebbe creare un vettore di 0, quante sono le categorie, a meno di un elemento del vettore che indicherà con 1 a quale categoria appartiene l'elemento(**one-hot encoding**).

Si può anche trasformare anche un testo in un numerico, dove ogni row conterrà il numero che una parola si ripete e le colonne a quale documento si riferiscono.

	Testo.txt	Text.txt
Ciao	2	0
Mondo	3	1
Introduzione	0	5
Al	2	4
Data-mining	3	5

Data cleaning

Durante il processo di raccolta, può verificarsi che le diverse fonti o il processo in sé possa generare degli errori. Gli errori hanno natura diversa, ovvero:

- **Valori mancanti:**
 - Si possono eliminare i record contenenti i valori mancanti(se vi fossero molti record con valori mancanti, dovremmo ripetere l'operazione di raccolta).
 - Stimare valori mancanti(media, mediano, valore vicino,ecc.), attenzionando che stime errate influenzano i risultati del data mining.
 - Lasciare decidere all'algoritmo -> potrebbe ignorare, fare stime o qualsiasi operazione prevista al suo interno.
- **Valori errati:**
 - Nell'integrazione da sorgenti diverse, si potrebbero avere problemi di formato, dominio, ecc., che viene rilevato. La rilevazione prevede anche il rilevamento dei dati duplicati.
 - La conoscenza del dominio è nota attraverso intervalli e regole tra le relazioni dei diversi attributi. Sulla base della conoscenza del dominio possiamo sviluppare strumenti di data scrubbing che rilevano voci errate. Es. Nazione = 'ITALIA', città = 'BORDEAUX' -> conoscendo il dominio potremmo rimuovere questo record
 - Attraverso metodi statistici dei dati, potremmo rilevare valori anomali(outliers) ed eliminarli. L'eliminazione di dati però potrebbe non essere sempre la soluzione migliore, dato che si possono rimuovere informazioni utili.

Per individuare gli outliers si può utilizzare una tecnica basata sui **quantili** (suddivisione di un insieme in n parti uguali). I **quantili di ordine x** (con $0 \leq x \leq 1$) è un valore **q_x**, che divide l'insieme dei valori S in due parti, x% elementi minori di q_x e (100-x)% di elementi maggiori di q_x.

Quando dividiamo in quattro parti abbiamo lavoriamo con i **quartili**. Il valore di ogni quartile si ricava dalla formula:

$$p_{qk} = (n+1) * k/4 \quad \text{con } k \in \{1, 2, 3\} \quad q_x = \text{interpolazione tra } p \text{ e } p+1 \text{ (attraverso la parte decimale)}$$

Avendo ad esempio l'insieme [-150, 0, 10, 20, 30, 40, 50, 80, 100, 1000]

$$p_{q1} = (10+1) * 1/4 = 2,75$$

$$q_1(25) = 0 + 0,75 * (10 - 0) = 7,5$$

$$p_{q2} = (10+1) * 2/4 = 5,5$$

$$q_2(50) = 30 + 0,5 * (40 - 30) = 35$$

$$p_{q3} = (10+1) * 3/4 = 8,25$$

$$q_3(75) = 80 + 0,25 * (100 - 80) = 85$$

possiamo adesso calcolare l'**interquartile range**, ovvero $q_3 - q_1$, che rappresenta l'ampiezza del 50% che riguarda la parte centrale dei dati.

$$IQR = q_3 - q_1 = 85 - 7,5 = 77,5$$

Un valore $x \in S$ è un outlier se:

- $x < q_1 - 1,5 * IQR$
- $x > q_3 + 1,5 * IQR$

verifichiamo per $x = -150$

$$x < 7,5 - (1,5 * 77,5)?$$

$$x < 7,5 - 116,25$$

$$x < -108,75$$

$-150 < -108,75$ sì, allora è un outlier

Scaling e normalizzazione

In molti casi, gli attributi hanno scale diverse e quindi non possono confrontarsi tra loro. Qualsiasi funzione di aggregazione applicata ai dati, risulterà essere fortemente influenzata dall'attributo con i valori più alti. Per questo motivo utilizziamo due diverse tecniche per sistemare le scale dei valori, standardizzandole per diversi attributi, rendendoli comparabili tra loro.

Le due tecniche sono:

- **standardizzazione**, operazione con la quale si riduce il fattore di scala. Si utilizzano la **media per un attributo μ** e la **varianza dell'attributo σ** $\sigma = \frac{\sum_{i=1}^n (x_i - \mu)^2}{n}$ (sostanzialmente si calcola la distanza media che si ha dalla media dei valori). Attraverso queste due variabili possiamo riassegnare il valore a z, ovvero: $x = \frac{z - \mu}{\sigma}$
- **min-max scaling**, operazione nella quale si inseriscono i valori di un attributo nel range [0, 1]. Si devono individuare massimo e minimo, inoltre x diventerà pari a $y = \frac{x - \min(S)}{\max(S) - \min(s)}$

Riduzione dei dati

L'obiettivo di ridurre i dati è quello di rappresentarli in modo più compatto. Infatti, quando i dati sono pochi è molto più facile applicare algoritmi molto costosi e sofisticati. La riduzione dei dati può avvenire o in termine di righe o in termini di colonne.

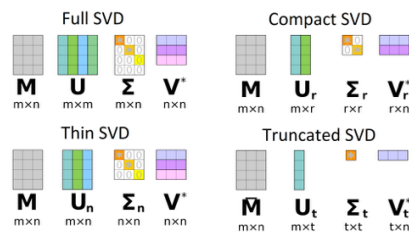
Anche se la riduzione dei dati comporta perdite d'informazioni, un algoritmo può compensare la perdita.

Le tecniche che si possono impiegare per la riduzione dei dati sono:

- **data sampling** -> si seleziona una porzione più piccola del dataset, che sia rappresentativo dell'intero dataset. Esso può essere semplice da eseguire in uno scenario statico, dove si hanno già tutti i dati, al posto di uno scenario dinamico, dove si dovrebbero considerare tutte le nuove entries. Inoltre, può essere di due tipi:
 - **biased**: alcuni record sono scelti con maggiore probabilità perché sono più significativi per l'analisi -> ad esempio nelle serie temporali gli eventi che sono stati catturati di recente.
 - **stratified**: si classificano i record e poi si campiona una porzione fissa di ogni classe -> si usa per mantenere alcune caratteristiche del dataset.
- **feature selection** -> alcuni feature si potrebbero rimuovere perché irrilevanti al fine del data mining. In che modo avviene la decisione? Ci sono due tipi di feature:
 - unsupervised feature selection: corrispondono agli attributi rumorosi o di ridondanza dei dati.
 - supervised feature selection: si rimuovono tutte le feature che non aiuta a effettuare predizioni utili.
- **riduzione di dimensionalità con rotazione di assi** -> In molti dataset reali alcuni dati sono correlati ad altri presenti nel dataset stesso, ed è per questo che si ha ridondanza, anche se implicita. Nel caso di due feature, dove la prima è l'età e la seconda l'anno di nascita, potremmo anche evitare di usare l'età, dato che è perfettamente ottenibile dalla data di nascita. Queste correlazioni possono anche essere implicite e complesse, per questo si deve fare un'analisi molto dettagliata. Si possono rimuovere anche alcuni attributi con la rotazione degli assi. Infatti, se si nota che un attributo è fortemente influenzato da un altro, basta utilizzare la tecnica della rotazione degli assi. Si pone quindi un asse nel grafico lineare e un asse descrive la varianza. Se la varianza è quasi uguale a 0 (o 0), allora si può eliminare l'asse che descrive la varianza. Le tecniche utilizzate per la riduzione di dimensionalità sono:
 - **principal component analysis** -> si ruotano i dati in un nuovo sistema di coordinate in cui la maggior quantità di varianza è catturata da un numero ridotto di dimensioni.
La varianza dei dati lungo una direzione è espressa dalla matrice di covarianza.
La matrice di covarianza, lungo la diagonale principale, contiene solo la varianza delle feature, mentre negli elementi i, j con $i \neq j$ sono riportate le covarianze tra due feature.
Se i valori di X corrispondono ai valori di Y, e vale lo stesso per i valori più piccoli, allora la covarianza è positiva.
Se il segno della covarianza è positivo, allora i due vettori sono correlati.
Gli elementi della matrice si calcolano così: $c_{ij} = \frac{1}{n-1} * \sum_{k=1}^n (x_{k,i} - \mu_i) * (x_{k,j} - \mu_j)$.
Sostanzialmente si scorrono tutte le righe sommando i valori con la media dei valori sottratta.
$$\begin{matrix} var(X,X) & covar(X,Y) & covar(X,Z) \\ covar(Y,X) & var(Y,Y) & covar(Y,Z) \\ covar(Z,X) & covar(Z,Y) & var(Z,Z) \end{matrix}$$

Di seguito, si calcolano gli autovalori e gli autovettori della matrice di covarianza e si trovano le componenti principali: autovalori grandi indicano maggiore varianza catturata e gli autovettori danno la direzione delle componenti, che sono non correlate tra loro.
Le componenti principali sono tante quante le dimensioni dei dati, ma si considerano solo quelle con la varianza maggiore.
Infine, avendo la matrice P_p (con N righe e P colonne) contenendo solo gli autovettori selezionati e la matrice D (con M righe e N colonne), ossia il dataset di partenza: $D' = DP_p$ (MxP)

- **single value decomposition** -> questa tecnica permette di decomporre la matrice M di dimensioni $n \times d$ in tre matrici:
 - **U** -> come ciascuna osservazione del dataset si comporta rispetto ai pattern principali, matrice $n \times n$ contenente gli auto-vettori di MM^T
 - **Σ** -> quanto ogni pattern contribuisce alla varianza totale; matrice diagonale $n \times d$ contenente lungo la diagonale principali gli autovalori di MM^T ;
 - **V^T** -> combinazioni lineari dei pattern principali; matrice $d \times d$ contenente gli auto-vettori $M^T M$



Si hanno diverse versioni della SVD, versioni più ridotte che potrebbero escludere elementi superflui.

Thin SVD -> taglia al minimo U e Σ -> decomposizione esatta;

Compact SVD -> rimuove tutte le righe di Σ che contengono valori nulli e di conseguenza anche di U e V^T -> decomposizione esatta;

Truncated SVD -> mantiene solo le righe di Σ che contengono i valori più alti singolari e rimuove tutto il resto -> decomposizione non esatta (approssimata).

La SVD produce due set di auto-vettori, uno per le feature (attributi) e uno per le osservazioni, a differenza della PCA.

PCA è solo un caso particolare della SVD, infatti se la PCA esamina la varianza o distribuzione rispetto alla media (si centra la media e si pone $= 0$), la SVD esamina la distribuzione rispetto all'origine $(0,0)$.

Il **latent semantic analysis** è un'applicazione della SVD al text mining. Si parte da una matrice di partenza M con n righe contenenti i documenti e d colonne contenenti i termini. Ogni elemento della matrice conterrà le frequenze normalizzate del termine in un documento specifico.

Si mantengono solo i pattern più significativi, ovvero una combinazione lineare di termini che tende a comparire spesso nei testi esaminati. Si riduce così significativamente la dimensionalità del dataset, ottenendo risultati simili alla PCA.

Dopo che si applica la riduzione dei dati si possono effettuare trasformazioni dei dati, in tipi meno complessi da analizzare. Un esempio visto sono le serie di Fourier, che attraverso semplici combinazioni di funzioni sinusoidali, si ottengono segnali più facili da analizzare.

Insiemi frequenti e regole d'associazione

La **market-basket analysis** (analisi del cestino della spesa) cerca di estrarre relazioni tra **item** ossia elementi singoli e basket ovvero delle transazioni.

Un basket è costituito da un insieme di item "acquistati" insieme o **itemset**. L'obiettivo dell'analisi è trovare insiemi di item che sono osservati insieme nei vari basket, itemset frequenti.

Possiamo definire il **supporto di un itemset** come il numero di basket in cui l'itemset è presente.

Nei nostri progetti potremmo fissare una **soglia di supporto σ** , ovvero una soglia che ci permetterà di stabilire che un itemset i può essere considerato frequente. In particolare, l'itemset i verrà considerato un itemset frequente se il suo supporto è maggiore o uguale alla soglia di supporto stabilita.

La scelta della soglia del supporto influenza i risultati trovati e determina la qualità degli itemset:

- se la soglia scelta è troppo bassa, il numero di itemset frequenti cresce rapidamente, implicando un'esplosione di soluzioni;
- se la soglia scelta è troppo alta si potrebbero perdere soluzioni importanti.

Una regola comunemente adottata per la scelta della soglia prevede nel fissarla pari all'1% dei basket.

Le applicazioni della market-basket-analysis sono estese a vari campi:

- **Marketing:** si cerca prodotti acquistati frequentemente insieme e si definiscono strategie promozionali, sconti e ottimizzazioni di prodotti nei punti di vendita;
- **Plagiarismo:** se gli item sono parole e i basket frasi, potremmo verificare se un testo è stato plagiato.

La ricerca di itemset frequenti è generalmente finalizzata allo sviluppo di regole di associazione, espressa nella forma di implicazioni (if-then).

Una regola di associazione si denota come:

$$I \rightarrow j$$

Dove:

- I è un itemset;
- j è un elemento singolo.

L'interpretazione della regola è la seguente:

Nei basket in cui si osserva l'itemset I , è probabile trovare l'item j . La probabilità è data dalla confidenza della regola, che misura la forza dell'implicazione valutando quanto spesso, quando compare I , compare anche j .

La confidenza della regola $I \rightarrow j$ è definita come:

$$Conf(I \rightarrow j) = Supp(I \cup \{j\}) / Supp(I)$$

Dove:

- $Supp(I \cup \{j\})$ indica la percentuale di basket dove compare sia I che j -> supporto della regola;
- $Supp(I)$ indica la percentuale di basket dove compare I -> coverage della regola.

Ad esempio, abbiamo due elementi pane e burro e vogliamo vedere se $\text{pane} \rightarrow \text{burro}$, allora avremmo che:

- $Supp(\text{pane} \cup \text{burro})$ = si contano la presenza nei basket di {pane,burro,...}
- $Supp(I)$ = si contano le presenze nei basket di {pane}

La confidenza è utile se il supporto di I è sufficientemente alto. Se ad esempio il supporto di I osservato è pari al 0.01% allora avremmo un valore di confidenza alto, che potrebbe risultare poco interessante.

Anche se il supporto di I è sufficientemente elevato in alcuni contesti, la regola potrebbe non risultare interessante. Ad esempio una regola: {pasta, pomodoro}.

Questa misura però non coglie quanto un itemset influenza un item. Per calcolare quanto un itemset influenza l'item si utilizza l'**interesse**. La formula dell'interesse di una regola $I \rightarrow j$ è la seguente:

$$Int(I \rightarrow j) = Conf(I \rightarrow j) - Supp(j)/N$$

dove N è il numero di baskets.

- Se $int > 0$ allora siamo in presenza di un'influenza positiva;
- Se $int = 0$ non c'è alcuna influenza;
- Se $int < 0$ c'è un'influenza negativa.

Un'alternativa alla misura dell'influenza dell'itemset sull'item è il **lift(indice di sollevamento)**.

Il lift è una misura che serve a capire se una regola di associazione è realmente utile oppure no. Infatti, viene effettuato il rapporto tra il supporto osservato e il supporto atteso nel caso in cui siano indipendenti I e j. La formula è la seguente:

$$Lift(I, j) = \frac{N * Supp(I \cup j)}{Supp(I) * Supp(\{j\})}$$

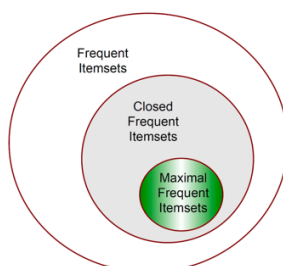
- Se il lift > 1 allora I esercita un'influenza positiva su j -> più alto è il valore, maggiore influenza esercita;
- Se il lift $= 1$ allora j è indipendente da I;
- Se il lift < 1 allora I esercita un'influenza negativa su j, più basso è il valore, maggiore influenza esercita;

Riassunto delle stime:

Stima	Breve descrizione	Formula
Confidenza	Conta quanto spesso l'item j compare nei basket dove è contenuto l'itemset I.	$Conf(I \rightarrow j) = Supp(I \cup \{j\}) / Supp(I)$
Interesse	Conta quanto la presenza di I cambia la probabilità di trovare l'item j	$Int(I \rightarrow j) = Conf(I \rightarrow j) - Supp(j)/N$
Lift	Verifica se c'è un'indipendenza tra I e j	$\frac{N * Supp(I \cup j)}{Supp(I) * Supp(\{j\})}$

Esistono anche altre varianti del problema, dove si ricercano:

- **insiemi frequenti chiusi**: ovvero non esiste un super-insieme (insieme che lo contiene) che ha lo stesso supporto;
- **insiemi frequenti massimali**: non esiste un super-insieme frequente che li contiene;
- **insiemi frequenti chiusi e massimali**: non esiste un super-insieme che ha lo stesso supporto e che non è frequente.



- Un itemset frequente chiuso è anche un insieme frequente.
- Un itemset frequente massimale è anche un insieme frequente chiuso.

Un algoritmo in grado di trovare un itemset frequente è anche in grado di trovare itemset chiusi e massimali. Questo perché itemset chiusi e massimali appartengono all'insieme dei itemset frequenti.

Se rappresentiamo il livello L_k con una tabella con k righe, l'insieme L_{k+1} può essere rappresentato attraverso una join L_k con sé.

Possiamo effettuare un'ottimizzazione dell'algoritmo Apriori attraverso l'**algoritmo PCY**, ovvero il **raggruppamento in bucket**. Per effettuarlo si prosegue nel seguente modo:

- 1) Per ogni itemset della dimensione k , calcoliamo la **funzione hash** che lo assegna in un bucket;
- 2) Ogni bucket tiene il conto di quante volte vi cade un item-set;
- 3) Se il conteggio del bucket sotto la soglia minima, allora gli elementi all'interno del bucket possono essere scartati;
- 4) Gli itemset il cui bucket ha superato la soglia, verranno considerati per il livello L_{k+1} ;

Questo metodo è molto veloce perché riduce il numero di item-set da contare effettivamente.

Un altro algoritmo che può gestire dataset molto grandi è l'**algoritmo SON** o **partizionamento del DB**, ovvero:

- 1) Si divide il dataset in tante partizioni dette **chunk**;
- 2) Ogni chunk ha una percentuale p di basket dell'intero dataset;
- 3) Si calcola la soglia minima attraverso $p \times s$, dove: p è un numero compreso tra 0 e 1 e s è la soglia minima del dataset;
- 4) Si creano itemset all'interno di ogni chunk e si verifica la soglia minima;
- 5) Infine, vengono prelevati gli itemset più frequenti e si confronta il supporto con la soglia originale;

Questo algoritmo è ottimale in architetture distribuite, dove ogni macchina cerca gli itemset frequenti di uno o più chunk.

Un altro algoritmo che utilizza il partizionamento del db al fine di trovare gli itemset frequenti è l'**algoritmo di Toivonen**. Esso non controlla tutte le partizioni bensì utilizza una partizione come campione da analizzare.

Importante in questo algoritmo è il concetto di **frontiera negativa**, ovvero l'insieme degli itemset che non sono frequenti, ma cui gli immediati itemset (ottenuti togliendo un solo item) sono frequenti.

Questi itemset appartenenti alla frontiera negativa potrebbero essere eventuali falsi negativi, cioè itemset che non sono frequenti nel campione casuale ma lo sono del dataset.

La soglia minima di S è pari a $\sigma \times p \times s$, dove:

- **σ è il parametro di sicurezza** compresa nell'intervallo $[0,1]$, che serve a ridurre la soglia minima del campione;
- **s è la soglia minima del dataset;**
- **p è il numero di basket nel campione.**

Si trovano gli itemset frequenti in S e quelli appartenenti alla frontiera negativa.

- Se gli elementi appartenenti alla frontiera negativa non sono frequenti, abbiamo trovato tutti gli itemset frequenti e abbiamo finito;
- Se gli elementi appartenenti alla frontiera negativa sono frequenti sul dataset, allora si riprova l'algoritmo su un nuovo campione.

Si presuppone che il campione S sia rappresentativo del dataset, quindi, deve essere sufficientemente grande e casuale.

L'approccio Apriori è inefficiente perché:

- ad ogni passo k , genera tutti i possibili itemset di lunghezza k ;
- scansa il database per verificarne il supporto.

Ciò vuol dire che quando k cresce o la soglia minima è bassa, il numero di candidati aumenta sprecando molta memoria e avendo un costo computazionale alto. Si possono trovare itemset frequenti senza generare candidati?

La principale alternativa all'Apriori è l'algoritmo **FP-Growth(frequent pattern grow)**, che permette di trovare itemset frequenti attraverso la crescita di itemset più piccoli frequenti.

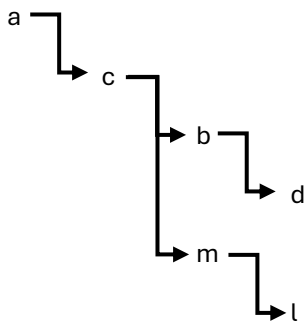
Invece di generare candidati, costruisce una **struttura ad albero compatto** che rappresenta tutte le transazioni del dataset, chiamata **FP-Tree**.

Le transazioni condividono i prefissi comuni e utilizzando l'FP-tree si ha:

- letture minori del dataset;
- risparmio della memoria;
- lavoro ridotto per la ricerca di item frequenti.

Attraverso l'utilizzo dell'FP-tree si ha un approccio divide et impera del problema, ovvero che si decompone in problemi più piccoli.

Se si hanno due o più transazioni che condividono un itemset frequente, la **porzione condivisa viene memorizzata una volta sola** -> ad esempio (a,c,d,b),(a,c,m,l) allora (a,c) sarà la porzione condivisa una sola volta.



L'algoritmo si costruisce di due fasi:

- costruzione dell'FP-tree;
- ricerca degli itemset frequenti.

Costruzione dell'FP-tree

Durante la costruzione dell'FP-tree si scansiona il DB, per creare la lista contenente gli item(elementi singoli) frequenti ordinata in modo decrescente(dal più frequente al meno frequente).

Viene creata la **root**(non contiene item) e partendo da essa si aggiunge il cammino, ovvero i nodi che verranno attraversati(si parte ad inserirli in modo decrescente). Se un nodo non è presente si aggiunge settando il contatore a 1, altrimenti si incrementa il contatore già presente.

Si collega l'FP-tree ad una tabella hash chiamata **header-table**, in cui:

- le **chiavi** sono gli **item frequenti**;
- il **valore associato** alla chiave è un **puntatore al primo nodo aggiunto** nell'FP-tree associato a quella chiave.

Ogni elemento quindi conterrà etichetta, valore e il node-link, ossia il puntatore al nodo successivo con la stessa etichetta.

L'FP-tree è una struttura dati:

- **completa**: rappresenta l'intero DB;
- **compatta**: il numero di nodi è al massimo pari al numero di occorrenze degli item frequenti del DB e la sua altezza è pari al massimo pari al numero di item frequenti nel DB

Ricerca degli itemset frequenti

La ricerca degli itemset frequenti avviene usando l'FP-tree e i puntatori node link. Per ottenere tutti gli itemset che contengono un elemento X si prosegue in due steps:

- si parte dall'hash table e si passa in tutti i nodi etichettati con X ;
- per ogni nodo X si segue il relativo cammino che porta alla radice.

Infine, per ogni nodo si ha il contatore che ci da informazioni sul supporto e se è maggiore della soglia, l'itemset trovato è frequente. Il supporto di una combinazione è dato dal contatore più piccolo di item del cammino.

Il vantaggio che si ha è relativo al non scorrimento di tutto l'albero, ma soltanto dei nodi etichettati con X .

Clustering

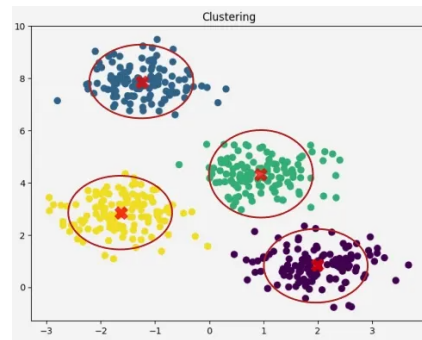
Il **clustering** è un processo che prevede di **raggruppare** un insieme di oggetti simili tra loro all'interno di un **cluster**. È un processo che serve a trovare una struttura naturale nei dati.

Nello stesso cluster quindi troveremo oggetti simili, invece, in cluster diversi troveremo oggetti dissimili tra loro.

Il clustering è un processo **non supervisionato**, dato che si vuole suddividere un insieme di dati in n classi, senza sapere a priori quanti e quali sono le classi e le etichette.

La **classificazione**, invece, è un processo supervisionato dato che si conoscono a priori le classi e le etichette associate. Quindi si addestra un modello a riconoscere i prossimi oggetti e ad inserirli all'interno delle classi.

Gli oggetti da raggruppare sono punti appartenenti a uno **spazio metrico S**, dove è possibile definire una **distanza(D)** per raggruppare i punti.



Gli attributi di un punto rappresentano le coordinate per individuarlo nello spazio. Gli attributi devono rispettare le seguenti proprietà:

- $D(X,Y) \geq 0 \quad \forall X, Y \in S$ in uno spazio reale non ci possono essere distanze minori di 0
- $D(X,Y) = D(Y,X) \quad \forall X, Y \in S$ (**proprietà simmetrica**) la distanza tra due oggetti è simmetrica, ovvero $D(\text{Roma}, \text{Milano}) = D(\text{Milano}, \text{Roma})$
- $D(X,Y) + D(Y,Z) \geq D(X,Z) \quad \forall X, Y, Z \in S$ (**proprietà triangolare**)

Un esempio di spazio metrico è lo spazio euclideo, dove si può calcolare la distanza tra due oggetti tramite diverse metriche, tra cui:

- **distanza euclidea;**
- **distanza di manhattan;**
- **norma L_r ;**
- **norma L_∞ ;**
- **distanza del coseno.**

In uno spazio euclideo la media è sempre definita per un insieme di punti, ed è un punto nello spazio chiamato **centroide**. In uno spazio non euclideo il concetto di centroide non è definito.

Se si ha uno spazio non euclideo però metrico si può definire il concetto di **medoide**, ovvero il **punto che minimizza la distanza media**(somma) **delle distanze dagli punti dell'insieme**(il punto più vicino a tutti gli altri).

Ad esempio, abbiamo 3 città A,B,C e la tabella delle distanze definita come segue:

COPPIA	DISTANZA
A-B	3
A-C	6
B-C	4

Vediamo quale delle città funge da medoide:

$$A = 3+6 = 9$$

$$B = 3+4 = 7$$

$$C = 6+4 = 10$$

B funge da medoide dato che è il punto più vicino a tutti gli altri(il punto che minimizza la distanza media delle distanze dagli altri punti).

Possiamo calcolare varie distanze in spazi metrici non euclidei e ognuna è utile a seconda del contesto.

La **distanza di Jacard** esprime la misura di **dissimilarità tra due insiemi**, quindi indica quanti oggetti sono diversi. La formula per calcolare la distanza di Jacard è la seguente:

$$J(S,T) = 1 - \frac{|S \cap T|}{|S \cup T|}$$

Esempio:

$A = \{1,2,3,4,5\}$

$B = \{1,4,6,7,8,9,10\}$

$A \cap B = \{1,4\}$

$$J(A,B) = 1 - \frac{2}{10} = \frac{8}{10}$$

La **distanza di edit** è il numero di operazioni di inserimento o rimozione caratteri da effettuare partendo da una stringa A per arrivare ad una stringa B.

Esempio:

A = "ciaoinformatici"

B = "ciaoinfospastici"

Operazioni:

- 1) cancella r
- 2) cancella m
- 3) inserisci s dopo la seconda o
- 4) inserisci p dopo s
- 5) inserisci s dopo la seconda a

$$D(A,B) = 5$$

La **distanza di Hamming** tra due vettori è il numero di componenti in corrispondenza delle quali i due vettori differiscono.

$$V_1 = (1,0,1,0,0,1)$$

$$V_2 = (1,0,0,1,1,1)$$

$$D(V_1, V_2) = 3$$

Possiamo effettuare diversi algoritmi di clustering che permettono di suddividere gli elementi nei cluster di appartenenza. Esiste una tassonomia(classificazione) dei principali tipi di algoritmi di clustering basata sull'approccio utilizzato e di seguito viene proposta.

- 1) **Metodi gerarchici o agglomerativi**: ogni punto inizialmente è un cluster e successivamente i cluster vengono combinati tra loro secondo la nozione di vicinanza definita-> le fusioni tra cluster sono costose soprattutto in presenza di più punti ;
- 2) **Metodi di partizione**: si sceglie a priori il numero di cluster k, e si assegnano i punti ad ogni cluster, cercando di minimizzare una misura d'errore(la distanza dei centri tipicamente)-> veloce ed efficienti anche in presenza di più punti -> rigidità dell'uso di un numero fisso di cluster e sensibile agli outlier e i primi valori immessi;
- 3) **Metodi basati sulla densità**: un cluster è una zona densa di punti separata da zone dense a zone meno dense e i punti all'interno delle zone vengono raggruppati sotto lo stesso cluster. Le zone si estendono fino ad una certa soglia.-> riconosco cluster di qualsiasi forma e non sono sensibili agli outlier -> il problema è legato alla soglia da scegliere.

La **bontà** di un algoritmo è legata a diversi fattori, non tutti necessariamente presi in esame quando si deve affrontare un problema. I fattori potrebbero essere: **scalabilità**, **capacità di ricercare cluster di forma diversa**, **capacità di gestire valori outlier e rumore** e **insensibilità all'inserimento di nuovi dati**.

In spazi euclidei ad elevata dimensionalità ($x_1, x_2, \dots, x_n$) sono affetti dai seguenti problemi:

- Quando n cresce, le distanze diventano simili tra loro;
- Le direzioni tendono ad essere diverse e i vettori ortogonali e il loro prodotto scalare è 0;
- La nozione di vicinanza perde di significato. -> la distanza diventa un concetto irrilevante

Quindi, i vettori di 2/3 dimensioni tendono a puntare in direzioni simili (ergo prodotto scalare $\neq 0$), se hanno dimensioni pari a 100 o simile tendono a puntare in direzioni diverse (prodotto scalare = 0).

Ricordiamo che la distanza euclidea si calcola come segue:

$$D(X, Y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Normalizzando i valori da 0 a 1 e avendo molte componenti nei vettori si ha:

- Se il valore massimo è 1, allora $D(X, Y)$ è almeno pari a 1, perché ci sarà almeno una componente x_n la cui differenza con una sua relativa produce risultato 1. Quindi individuiamo un limite inferiore.
- Se la differenza tra tutte le componenti produce 1, allora $D(X, Y)$ è pari $\sqrt{n}(\sqrt{1^2 + 1^2 + \dots + 1^2})$. Si individua quindi un limite superiore, ma la maggior parte delle distanze sarà pari a $\frac{\sqrt{n}}{3}$ (tipicamente $\approx \frac{1}{3}$ se i dati sono uniformi).

Clustering gerarchico

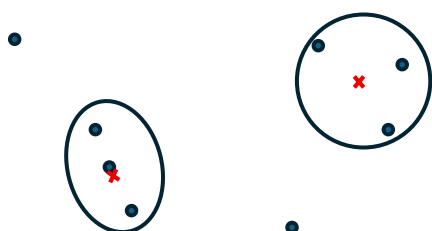
Come citato prima, il clustering gerarchico è una tipologia di clustering che procede nel seguente modo:

- Assegna ciascun punto ad un cluster;
- Unisce i due cluster più vicini in un unico cluster (definita la distanza);
- Si ripete la fusione finché non si arriva alla terminazione.

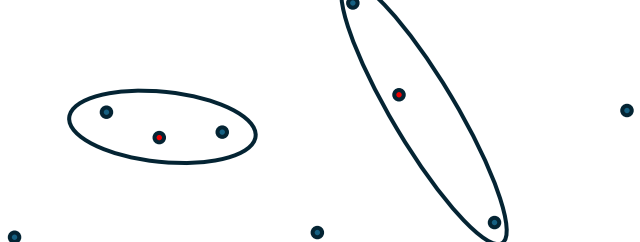
Nel dettaglio vediamo come:

- **In uno spazio euclideo** -> si considera il centroide (media delle coordinate dei punti) di ogni cluster (nel caso di un solo punto all'interno del cluster allora viene preso il punto stesso) che solitamente è un punto ideale, si calcola la distanza con gli altri centroidi e infine il cluster che ha il centroide più vicino viene fuso al cluster considerato.
- **In uno spazio non euclideo** -> (concetto di centroide non esiste) si prende solitamente un clustotide chiamato poi medoide (punto reale del dataset) perché è quello che ha la somma delle distanze dagli altri clustotidi minima. Si calcola la distanza con altri medoidi e si fa la fusione con il cluster che ha il medoide più vicino rispetto a quello preso in considerazione.

Spazio euclideo

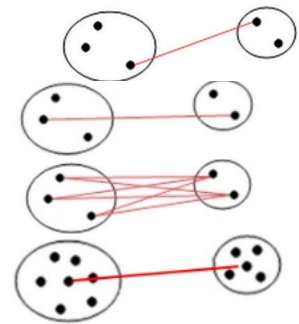


Spazio non euclideo



Oltre la distanza minima tra centroidi o medoidi, si può considerare anche:

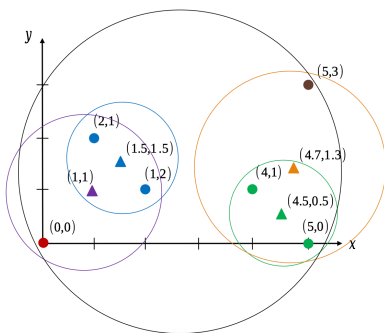
- **Single link** -> si considera la distanza minima tra un elemento appartenente al cluster X e un elemento appartenente al cluster Y;
- **Complete link** -> si considera la distanza massima tra un elemento appartenente al cluster X e un elemento appartenente al cluster Y;
- **Average link** -> considera la media di tutte le distanze tra tutte le coppie appartenenti a due cluster X e Y;
- **Medoid link** -> considera la distanza tra due medoidi di X e Y,



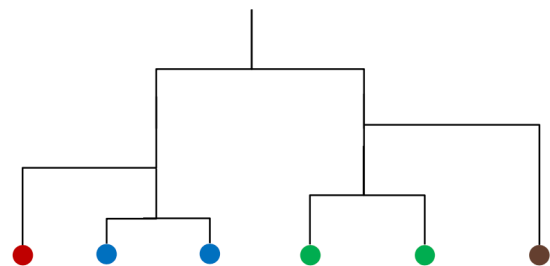
Il **dendrogramma** è un albero gerarchico che serve per rappresentare come i punti vengono uniti passo passo.

Nella rappresentazione abbiamo che:

- le **foglie** rappresentano i **singoli punti**;
- i **rami** rappresentano la **fusione tra cluster**;
- l'**altezza di un ramo** rappresenta la **distanza tra cluster**:
 - **altezza ridotta** = cluster vicini (distanza ridotta);
 - **altezza elevata** = cluster lontani (distanza elevata);
- **taglio dell'albero**, permette di vedere i **cluster finali, ad una certa altezza** (distanza):
 - **taglio alto** -> pochi cluster grandi;
 - **taglio basso** -> molti cluster piccoli.



DENDROGRAMMA CORRISPONDENTE



I criteri di terminazione per il clustering possono essere:

- **numero cluster raggiunto** -> conoscendo a priori il numero di cluster che il nostro algoritmo deve trovare, possiamo fermarci quando questo numero è stato raggiunto;
- **cluster inadeguato** -> la distanza media dei centroidi dei rispettivi cluster è elevata;
- **raggio del cluster** -> dato un cluster prodotto dalla fusione di due precedenti, la distanza dal centroide e un punto del cluster produce un raggio elevato (non adeguato);
- **diametro del cluster** -> dato un cluster prodotto dalla fusione di due precedenti, la distanza da un punto ad un altro punto del cluster produce un diametro elevato (non adeguato);

L'approccio del clustering gerarchico può essere diviso in due sottogruppi, pur mantenendo le metriche e i criteri visti prima. I due sottogruppi sono:

- **clustering agglomerativo** -> ogni punto forma un cluster inizialmente e poi i cluster più vicini vengono fusi (bottom-up);
- **clustering divisivo** -> tutti i punti sono posti nello stesso cluster, e ad ogni passo un cluster viene suddiviso in due, secondo criteri di separazione (top-to-down).

In ogni caso la complessità dell'algoritmo di clustering si aggira a $O(n^3)$ perché i confronti all'inizio hanno complessità pari a $O(n^2)$ e nei successivi passi si ha complessità $O(n - 1^2)$, $O(n - 2^2)$, ..., $O(n - m^2)$.

Possiamo ridurre la complessità utilizzando **le code di priorità**. Che permettono di cercare la distanza minima in tempo costante $O(1)$ e di fare cancellazioni di distanze tra due cluster o inserimenti in tempo $O(\log n)$, per questo motivo avremmo una prestazione pari a $O(n^2 \log n)$.

Clustering partizionale

I clustering partizionali sono una famiglia di clustering che lavorano su uno spazio euclideo e assumono la conoscenza a-priori del numero k di cluster.

I passi sono i seguenti:

- scelta dei centroidi;
- costruzione dei cluster mediante i centroidi ottenuti in precedenza;
- assegnamento punti ai cluster più vicini;
- ricalcolo centroide;
- spostamento (rietichettatura) dei punti che sono vicini a nuovi centroidi;
- si ripete finché non c'è un stabilizzamento.

Per trovare i centroidi, si può seguire un algoritmo greedy, ovvero:

- 1) si prende un punto casuale e si aggiunge all'insieme dei centroidi S ;
- 2) si prende il punto che è più distante e si aggiunge a S ;
- 3) si ripete il passo, finché $|S| \leq k$ (numero di cluster);

In questo algoritmo di clustering si cerca di avere la minima distanza da un punto rispetto al suo centroide. Per terminare l'algoritmo si deve cercare una stabilità nella riassegnazione dei punti ai nuovi centroidi. Il **criterio di arresto** si ottiene attraverso la funzione obiettivo legata alla distanza media tra i punti e il corrispettivo centroide.

La funzione è definita così:

$$J = \sum_{i=1}^K \sum_{X \in P_i} (X - c_i)^2$$

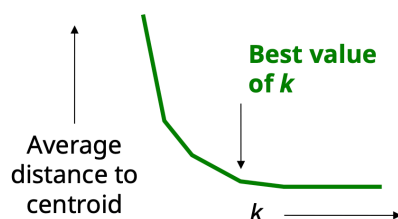
Fissata una soglia $\varepsilon > 0$, si termina quando:

$$J_{precedente} - J_{corrente} < \varepsilon$$

Ma come si sceglie il valore k ? Dobbiamo ottenere quello per cui il clustering risulta poi essere il migliore, e la qualità può essere misurata ad esempio attraverso la distanza media dei punti.

Il primo approccio da seguire è il **metodo Elbow** (gomito). Esso sfrutta la distanza media dei punti citata su e la mette in correlazione con k . Infatti, se noi abbiamo più cluster, allora gli elementi al proprio interno saranno vicini, se sono meno l'inverso.

Il valore ideale corrisponde al gomito della curva come mostrato nel grafico:



Dall'intervallo dei valori che può assumere k si sceglie il valore $z = \frac{(x+y)}{2}$, e si effettua il clustering per $k = z$. Si calcola la distanza media dei centroidi e:

- se per $k = z$ si ottiene un valore vicino a $k = x$ allora $y = z$ e si effettua nuovamente il calcolo;
- se per $k = z$ si ottiene un valore vicino a $k = y$ allora $x = z$ e si effettua nuovamente il calcolo.

Si ripetono i passi finché l'intervallo non è sufficientemente piccolo.

Un approccio più preciso è il **metodo silhouette**, che consiste nel calcolare che misura lo score di silhouette, ovvero la misura che indica quanto ciascun punto è ben assegnato al proprio cluster rispetto che ad altri.

Per ottenere lo score si devono calcolare:

- $a(X)$ -> ovvero la distanza media di X da tutti gli altri punti appartenenti allo stesso cluster;
- $b(X)$ -> ossia la distanza media di X da tutti gli altri cluster.

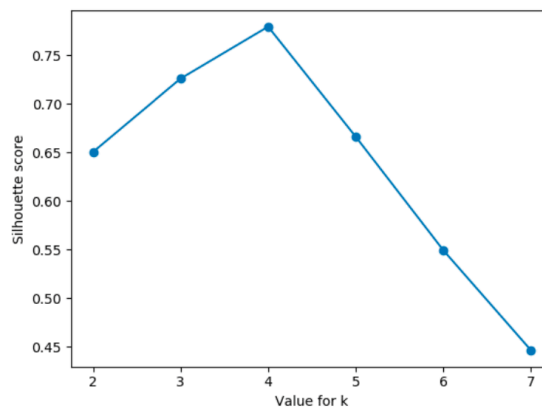
Il valore di silhouette di X verrà calcolato attraverso la seguente formula:

$$s(X) = \frac{b(X) - a(X)}{\max\{b(X), a(X)\}}$$

il valore è quindi compreso tra $[-1, 1]$ e se:

- $s < 0$ l'assegnazione di X a C_i è sbagliata;
- $s \approx 0$ l'assegnazione di X è un punto al confine tra cluster;
- $s > 0$ l'assegnazione di X a C_i è buona;

Lo score di silhouette si ottiene dalla media di tutti i punti e si sceglie lo score che ha valore massimo.



La complessità è pari a $O(tkn)$ con t il numero di round, k il numero di cluster e n il numero di elementi. t e k solitamente sono minori a n, per questo motivo è più efficiente del metodo gerarchico.

Cluster per densità

Infine, abbiamo il metodo **DBSCAN**(Density-Based Spatial Clustering of Applications with Noise) che prevede un algoritmo basato su densità.

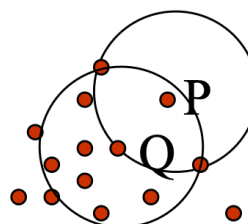
Ogni cluster è una **regione con una densità di punti sufficientemente alta** in un intorno sufficientemente limitato. DBSCAN ha due parametri:

- ϵ raggio legato alla grandezza del cluster;
- **MinPts** che rappresenta il numero di punti che il cluster deve avere.

Attraverso questi due parametri, possiamo quindi definire $N_\epsilon(Q)$, ossia l'insieme dei punti a distanza al più ϵ da Q.

In questo caso abbiamo $P \in N_\epsilon(Q)$ con Q che è un core-point, ossia $|N_\epsilon(Q)| \geq \text{MinPts}$.

In poche parole, Q è un punto popolare (ha tanti punti vicini, più del minimo previsto) e P è un punto vicino diretto.

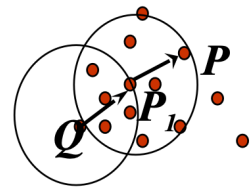


MinPts = 5
 $\epsilon = 1cm$

Un punto **P** è **raggiungibile per densità da Q** se e solo se esiste una catena di punti $Q, A_1, \dots, A_n, P$ tali che è raggiungibile per densità da A_n .

Le proprietà da soddisfare sono:

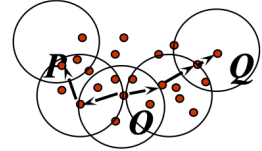
- $Q, \dots, A_n$ sono core-point, tranne P (può anche non esserlo);
- è un **cammino unidirezionale**.



Un punto P è connesso per densità a Q se esiste un punto O tale che sia P che Q siano raggiungibili da O.

Le proprietà da soddisfare sono:

- deve essere un core-point;
- i cammini sono $O \rightarrow P$ e $O \rightarrow Q$;



Arriviamo ora alla definizione di cluster, infatti, in DBSCAN un cluster è un insieme massimale di punti connessi per densità. Quindi avendo se l'insieme di punti da clusterizzare D e fissati ϵ e MinPts, allora C è un clustered se:

- $\forall P \text{ e } Q \in D$, se $P \in C$ e Q è raggiungibile per densità da P, allora $Q \in C$ (massimalità);
- $\forall P \text{ e } Q \in C$, P è connesso per densità a Q (connettività).

L'algoritmo DBSCAN funziona come segue:

- 1) Si sceglie un punto P randomico;
- 2) Si calcola l'intorno ϵ di P e si verifica se contiene almeno MinPts. Se sì allora si crea un nuovo cluster C e si prosegue, altrimenti si torna al punto iniziale (perché probabilmente il punto preso è un outlier);
- 3) Si aggiunge P e il suo intorno a C;
- 4) Si aggiungono i punti raggiungibili per densità di P, finché è possibile;
- 5) Ritorna al passo 1 fin quando tutti i punti non sono stati visitati.

Come si scelgono i parametri?

Per **MinPts**, si ha una nozione, infatti se si ogni punto è caratterizzato da **n attributi** segue che **MinPts** $\geq n+1$. Si fa così per stimare una densità vera, non un allineamento casuale.

Quindi:

- 1 dimensione, servono almeno 2 punti per occupare un intervallo;
- 2 dimensioni, servono almeno 3 punti per occupare un'area;
- 3 dimensioni, servono almeno 4 punti per occupare uno spazio.

Per trovare ϵ si utilizza il metodo **k-distance**, ossia per ogni punto si trova la distanza del k-esimo punto più vicino, dove $k = \text{MinPts}$. Si costruisce un grafico a due dimensioni, dove:

- sull'asse delle x sono presenti i punti;
- sull'asse delle y sono presenti le distanze ordinate in modo decrescente;

Attraverso la regola del gomito (Elbow) si trova il punto ottimale. Inoltre, scegliendo un ϵ grande si ottengono pochi cluster molto grandi, viceversa piccoli cluster e alcuni punti non clusterizzati.

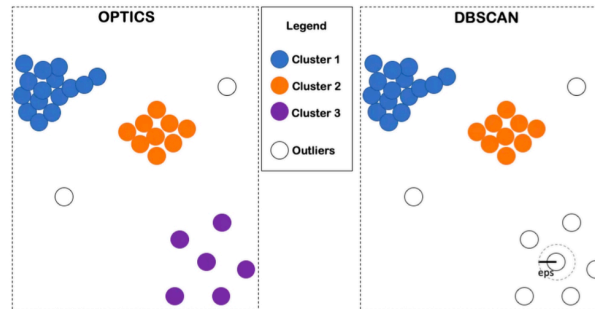
La complessità di questo algoritmo è pari a **$O(n \log n)$** attraverso strutture indicizzate come R-tree.

METODO	COMPLESSITA'	FORME TROVATE	CONTRO
Gerarchico	$O(n^3)$ riducibile a $O(n^2 \log n)$	Arbitrarie, dipende dalla metrica utilizzata	Molto costoso
Partizionale	$O(t \cdot k \cdot n)$	Sferiche, convesse	Molto semplice, rigido e sensibile all'outliers.
Densità	$O(n \log n)$	Forme irregolari	Soffre della maledizione della dimensionalità

OPTICS

OPTICS(Ordinary Points to Identify Clustering Structure) è un algoritmo di clustering basato sulla densità che risolve il problema di identificazione di cluster a densità diversa.

Non si stima un valore ϵ , bensì il valore del raggio è calcolato sulla base della densità dei punti in una regione dello spazio.



Per capire se due punti sono vicini in un'area localizzata, bisogna capire il contesto di essa e stimare se sono sufficientemente vicini per essere raggruppati.

Per definire **un'area localizzata si deve avere un centro e un raggio**. Il centro è uno dei qualsiasi punti, invece, il raggio è ϵ che si prende relativamente alto, in modo da non esaminare un'area troppo piccola. **Un'area localizzata** merita di essere esaminata solo se contiene un numero minimo di MinPts(incluso il centro).

Per capire se due punti della stessa area sono vicini, basta esaminare le distanze più piccole tra i punti della stessa area. La **distanza di raggiungibilità** o **reachability-distance** si calcola nella seguente maniera:

$$reachability - distance = \max(core - distance(O), distance(O, P))$$

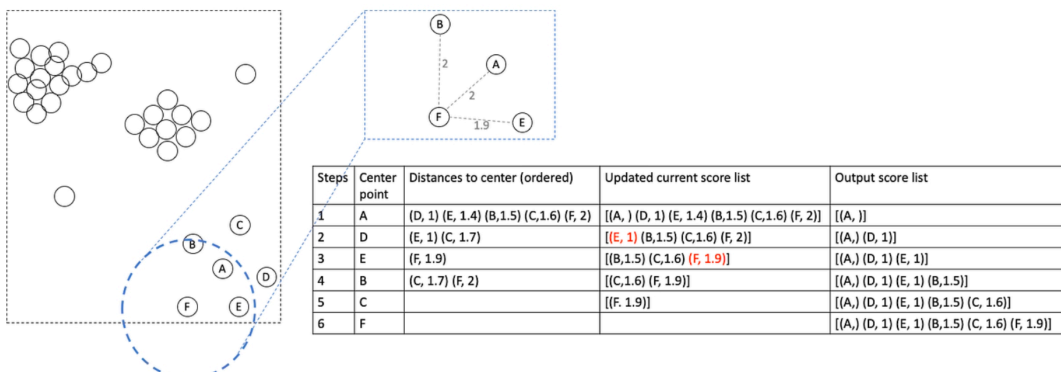
dove:

- $core - distance(O)$ è la distanza minima tale che il punto O abbia nel suo intorno MinPts;
- $distance(O, P)$ è la distanza euclidea tra O e P;

Se lo score è basso allora è più facilmente raggiungibile da altri punti del dataset.

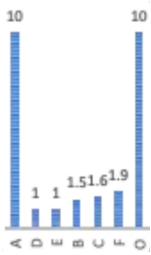
Per calcolare i reachability score si segue il seguente algoritmo:

- 1) Imposta il valore di reachability score di ogni punto con un numero molto alto;
- 2) Si prende P un punto del dataset;
- 3) Per ogni punto X dell'intorno si calcola la distanza da P, se la distanza $D(X, P)$ è minore di Reachability Score $R(X)$, allora si assegna al reachability score il valore di $D(X, P)$;
- 4) Il prossimo punto analizzato è quello con il reachability score più basso;
- 5) Si ripetono i passi 3 e 4 finchè non ci sono più punti da visitare.



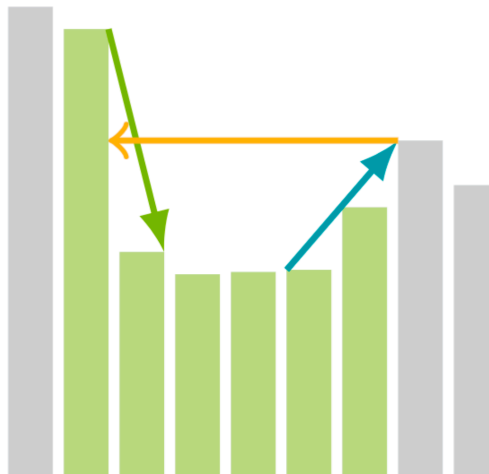
Con i valori trovati, li mettiamo in un istogramma dove:

- sull'asse delle x, vengono posti in ordine di visitazione i punti;
- sull'asse delle y, vengono posti i relativi rechability score.



Infine, per raggruppare il cluster, possiamo procedere in due metodi:

- si imposta una soglia ε e si scelgono tutti i punti con rechability-score $\leq \varepsilon$;
- si scelgono i punti estremi A e B, dove inizia e finisce l'avvallamento, e si prendono pure i punti al proprio interno, raggruppandoli in cluster (a patto che siano almeno MinPts).



Classificazione

Il clustering citato prima è una tecnica di apprendimento non supervisionato, quindi, partendo da un insieme di dati senza etichette o soluzioni cerca dei pattern, strutture o gruppi omogenei di cose.

La **classificazione** è una tecnica di **apprendimento supervisionato** e a partire da dati e da etichette **classifica gli oggetti in classi note** al modello.

Per creare un modello di classificazione, si parte da un insieme di dati chiamato **training set** e si allena un modello che è in grado di classificare correttamente i record.

Dopo che si è allenato un modello su dati conosciuti si fa un **test di validazione**, ossia si testa la bontà del modello, su un secondo set di dati chiamato **test set**. In questo modo si predicono i valori attraverso il modello e si confrontano con i dati del test set.

Se la validazione del modello da esito positivo, allora si potrà utilizzare il nostro classificatore per nuovi record.

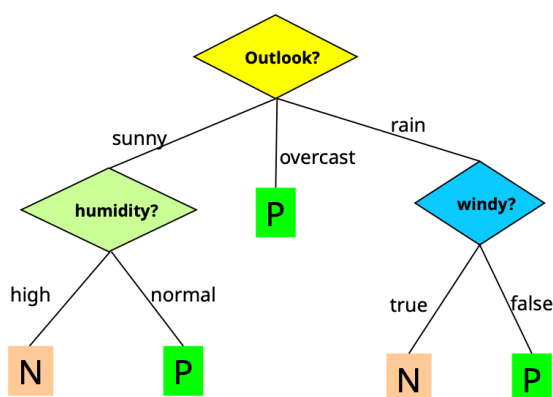
La **predizione(regressione)** permette di ricavare un **valore sconosciuto numerico continuo** a partire dagli altri attributi. Per trovare un predittore si deve creare un modello a partire dai dati del training set, che non cerca di separare dati di diverse classi, ma che deve **individuare una funzione matematica continua** che metta in relazione i dati e cerca di stimarne di nuovi.

I requisiti di un classificatore sono:

- L'**accuratezza** con cui predice correttamente le etichette delle classi o stima valori nuovi;
- La **velocità** con cui si costruisce un modello e per usarlo;
- La **robustezza**, ossia la capacità di manipolare i dati con rumore e mancanti;
- La **scalabilità**, ovvero la capacità di lavorare anche con tanti dati risiedenti in RAM o memorie secondarie.

Un'implementazione classica di classificatore è l'**albero decisionale**. Si utilizza una struttura ad albero(non necessariamente binario) per effettuare una serie di test sui valori degli attributi che predicono la classe di appartenenza di una tupla. L'albero decisionale è così come segue:

- Ogni **nodo** interno **contiene un test su un attributo**;
- Ogni **foglia** **contiene l'etichetta di una classe**;



Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

La **classe** si ottiene partendo **dalla radice** e **seguendo un path** che arriva **alla foglia**. Il nodo foglia come indicato prima determina la classe predetta.

Ogni path dell'albero applica regole **IF-THEN** per stabilire la classe di appartenenza e le condizioni sono legate attraverso l'**AND** logico. Due proprietà delle regole prodotte attraverso gli alberi decisionali:

- **Regole esaustive** -> per ogni combinazione di valori esiste una regola;
- **Regole mutualmente esclusive** -> per ogni combinazione di valori esiste una e una sola regola.

Per la costruzione di un albero decisionale si segue un **approccio top-down**, ossia si parte dalla radice e si trovano tutte le foglie. Possiamo anche dire che si segue un processo “divide et impera”: si parte dal problema principale (avere l’insieme di dati) e lo si spezzetta in problemi più semplici sino ad arrivare alle foglie per la classificazione.

L’algoritmo per la costruzione di un albero si può descrivere così:

- **STOP CONDITION:** se tutte le tuple della partizione S_x , associata ad un nodo interno X , hanno la stessa classe C allora si crea un nodo foglia con la classe associata C (si è già arrivati alla soluzione per quella classe);
- **CORPO CICLO:** si sceglie un attributo A non ancora considerato, si effettua una ramificazione di X (splitting) sulla base dei valori di A , creando due o più nodi figli;

Per ogni nodo figlio X_i di X :

- se X_i è **puro**, ossia che tutte le tuple di tale nodo hanno la stessa classe, passiamo ad un altro nodo;
- se X_i è **impuro**, si applicano ricorsivamente i passaggi del corpo del ciclo su X_i .

Se ci sono poche tuple associate al nodo X oppure l’albero ha raggiunto una profondità elevata, possiamo decidere di rendere **X una foglia, anche se impuro**.

Questo si fa per evitare **over-fitting**, ossia che l’albero impari a memoria anche i rumori, ergo sbaglierà su dati nuovi. Inoltre, se si stanno cercando regole universali non è necessario classificare due tuple ad una profondità elevata dell’albero.

Possiamo associare a X :

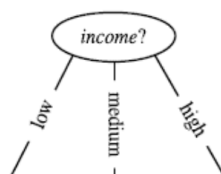
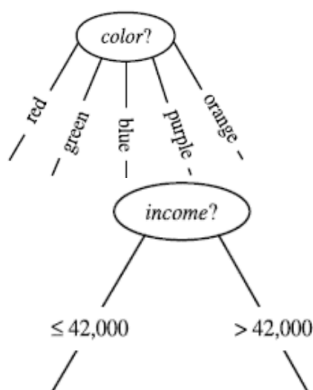
- l’etichetta della classe più frequente tra le tuple di X ;
- la probabilità con cui X possa assumere o un’etichetta o un’altra.

Come accennato prima l’albero decisionale non necessariamente deve essere binario, ma solitamente lo è. Questo perché la gestione binaria è preferita per motivi di efficienza e robustezza. Per attributi categoriali uno **split multiplo** crea subito tanti rami, disperdendo istantaneamente i dati -> albero largo e poco profondo.

Anche la gestione in memoria di una struttura dove ogni nodo ha esattamente 2 puntatori è molto più efficiente, invece che gestire un array di puntatori.

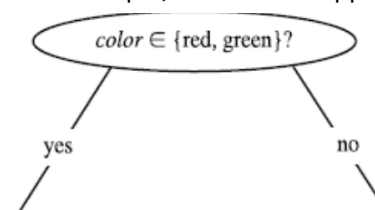
Per i motivi citati in precedenza, quando lavoriamo con attributi categoriali utilizzeremo esclusivamente **diramazioni binarie**. Si definisce un sottoinsieme di appartenenza e se:

- il valore dell’attributo non appartiene al sottoinsieme allora si procede a sinistra;
- il valore dell’attributo appartiene al sottoinsieme allora si procede a destra.

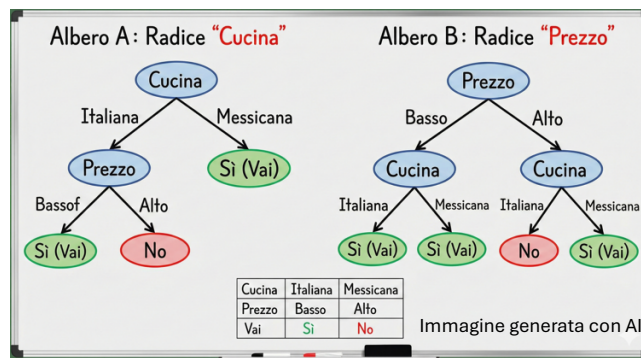


Per valori booleani o numerici si sceglie una soglia: se il valore è minore, allora si procede a sinistra altrimenti a destra.

Per categoriali si prende un sottoinsieme, come nell’esempio, e si verifica l’appartenenza.



La scelta degli attributi con cui si fa lo splitting influenza la costruzione degli alberi.



L'obiettivo è scegliere l'albero più semplice e compatto possibile, ossia l'albero minimale. La **scelta dell'albero minimale** ricade nei problemi **NP-hard**.

Come in altri problemi NP-hard, si utilizza una **strategia greedy**.

La strategia prevede che ad ogni passo si **selezioni l'attributo che produce le sotto-partizioni più pure**, ossia quelle che individuano in poche iterazioni un gruppo di elementi che già appartengono ad una classe.

Tutto ciò si effettua attraverso una misura di **goodness**, infatti **più alta è la goodness e più pure saranno le partizioni generate**.

Le misure di goodness utilizzate sono: l'**information gain**, **gain ratio** e **gini index**.

Information Gain

L'information gain seleziona l'attributo di splitting che mira alla riduzione progressiva dell'entropia. L'entropia è data dalle frequenze:

- se le frequenze sono simili per tutte le classi di un nodo si ha un'**entropia massima**;
- se i dati di uno nodo hanno quasi tutti la stessa classe si ha un'**entropia minima**.

Supponendo di avere due sole classi per un nodo dell'albero, ossia P e N. Inoltre, indichiamo con p il numero di tuple con classe P e n quelle di classe N.

L'entropia di S_x (insieme delle tuple del nodo X) è pari a:

$$H(S_x) = -\frac{p}{p+n} \cdot \log\left(\frac{p}{p+n}\right) - \frac{n}{p+n} \cdot \log\left(\frac{n}{p+n}\right)$$

Partendo sempre dalla formula principale:

$$E = -frequenza * \log(frequenza)$$

Possiamo calcolare l'entropia media considerando, per ogni valore dell'attributo A, la partizione S_i delle tuple in cui A assume quel valore. Si divide S in $\{S_1, S_2, \dots, S_k\}$ e si fa la media pesata delle entropie:

$$H(S|A) = \sum_{i=1}^k \frac{|S_i|}{|S|} H(S_i)$$

L'information gain è quindi definita come la riduzione di entropia ottenuta dal partizionamento di S_x scegliendo l'attributo A, ossia:

$$gain(A) = H(S_x) - \overline{H}_A(S_x)$$

Si sceglie come attributo quello che massimizza il gain.

Gain Ratio

Il problema principale nell'uso dell'information gain è che attributi con molti valori sono più favoriti. Per risolvere questo problema si è pensato di utilizzare il **gain ratio**, che misura quanto l'attributo A divide i dati (quanto è dispersiva la divisione).

Si calcola la **splitinfo**, che ci da informazioni su quanto i dati vengono divisi come segue:

$$splitinfo(A) = - \sum_{i=1}^k \frac{|S_i|}{|S_X|} \log_2 \frac{|S_i|}{|S_X|}$$

Se i dati sono divisi in piccole sezioni allora il valore dello splitinfo è grande.

Il gain ratio è calcolato come:

$$gainratio(A) = \frac{gain(A)}{splitinfo(A)}$$

Gli attributi che avranno molte sotto partizioni avranno un gain più basso.

Gini Index

Si considerano le n classi i e le tuple associate. Per ricavare il Gini Index occorre considerare:

la probabilità di scegliere una tupla S_x con classe i;

la probabilità di scegliere una tupla di classe j diversa dalla classe i.

$$gini(S_x) = \sum_{i=1}^n p_i(1 - p_i) = \sum_{i=1}^n (p_i - p_i^2) = \sum_{i=1}^n p_i - \sum_{i=1}^n p_i^2 = 1 - \sum_{i=1}^n p_i^2$$

misura la probabilità che i sia diverso da j.

Supponendo che S_x sia partizionato in k sottoinsiemi $S_1, S_2, \dots, S_k$. Il gini index è dato da:

$$gini_{split}(S_x) = \sum_{i=1}^k \frac{|S_i|}{|S_x|} gini(S_i)$$

L'attributo A scelto come attributo di splitting è quello che minimizza il **Gini split**.

Pruning di un albero decisionale

Il numero di attributi alto e i rispettivi valori in un training set fa crescere la dimensione dell'albero, e soprattutto si adattano ai dati di train e funzionano male su dati di test (overfitting).

Per questo motivo si effettua un **pruning**, ossia si **sostituisce un intero sottoalbero con un solo nodo radice etichettato** con la classe più frequente presente in tali tuple. Il tasso di errore introdotto dal pruning non deve essere elevato.

Il pruning può essere di 3 tipi, ossia:

- **pre-pruning**: in fase di costruzione dell'albero -> si blocca lo splitting se si hanno poche osservazioni, la profondità è elevata o il miglioramento in purezza è piccolo;
- **post-pruning**: l'albero già costruito viene analizzato e vengono rimossi rami e nodi, sostituendoli direttamente con delle foglie.

Il pre-pruning è veloce ma non è accurato, dato che potrebbe generare un albero troppo semplice troppo presto.

Per un pruning più accurato si sceglie il post-pruning che può essere eseguito in due diversi modi:

- **pruning pessimistico**: chiamato così perché un sottoalbero è soggetto a pruning sulla base dell'error rate, che viene **sovrastimato** attraverso ϵ . Si mettono a confronto quindi l'errore nella predizione quando il sottoalbero è stato sostituito con la radice e quando invece avviene lo splitting di X.

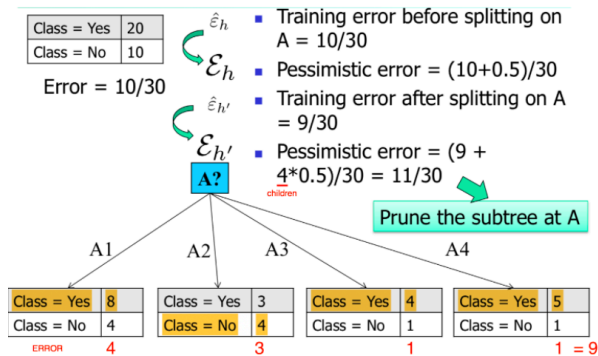
Per calcolare l'**error rate pessimistico** $E_p(T)$ prima di fare lo split di X su A:

$$E_p(T) = \frac{|\{t \in S_X : \text{class}(t) \neq C\}| + \epsilon}{|S_X|}$$

Per calcolare l'**error rate pessimistico** $E'_p(T)$ dopo aver fatto lo split di X su A:

$$E'_p(T) = \frac{\sum_{i=1}^k |\{t \in S_{X_i} : \text{class}(t) \neq C_i\}| + k\epsilon}{|S_X|}$$

dove k è il numero dei valori che può assumere l'attributo A.



- **cost-complexity pruning**: si calcola sempre l'error rate senza sovrastima prima dello splitting e dopo e si definisce una funzione chiamata **cost-complexity error**:

$$\alpha = \frac{E(T) - E'(T)}{k - 1}$$

se il valore è sufficientemente piccolo allo si può evitare lo splitting, ergo si pruna l'albero.

I vantaggi di questi classificatori sono:

- alta interpretabilità;
- molto veloci.

Gli svantaggi sono:

- poca accuratezza in presenza di tanti dati;
- overfitting.

Classificatori generativi

I **classificatori generativi** producono un **modello probabilistico** a partire dai dati, quindi, predicono **la classe di appartenenza più probabile**.

Essi si basano sul teorema di Bayes che afferma:

$$P(H_C|X) = \frac{P(X|H_C) \times P(H_C)}{P(X)}$$

Dove:

- **P(X)** è la probabilità di osservare la tupla X;
- **Probabilità a priori P(H_C)**: probabilità che l'ipotesi sia vera indipendentemente da X;
- **Likelihood P(X|H_C)**: probabilità di osservare X dato che l'ipotesi sia vera.
- **P(H_C|X)**: l'obiettivo, ossia la probabilità che l'ipotesi sia vera a partire dai valori osservati per gli attributi di X.

La classe C da assegnare è quella che massimizza P(H_C|X). I valori che si hanno nella formula si calcolano nel seguente modo:

- **P(H_C)** è la percentuale di tuple del dataset che hanno la classe C;
- **P(X)** si ignora, perché è costante rispetto le classi;
- **P(X|H_C)** è molto complesso da calcolare dato che è una probabilità congiunta tra variabili.

Assumendo (assunzione naïve) che $x_1, x_2, \dots, x_n$ siano indipendenti e fissata la loro classe C, allora:

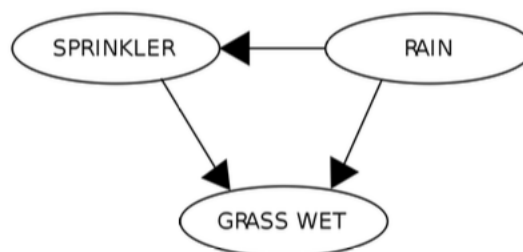
$$P(X|H_C) = P(x_1|H_C) \times P(x_2|H_C) \times \dots \times P(x_n|H_C)$$

Le probabilità condizionali risultano essere molto piccole, perché il prodotto è tra tante quantità prossime a 0. Questo può portare ad un **underflow computazionale**, per questo motivo si utilizza il **log-likelihood**, ossia:

$$\log P(X|C_i) = \log \prod_{k=1}^n P(x_k|C_i) = \log P(x_1|C_i) + \dots + \log P(x_n|C_i)$$

I vantaggi nell'utilizzare il naïve bayes sono legati alla semplicità e alla velocità di esecuzione, ma gli svantaggi sono legati alla non accuratezza del modello, dato che si assume indipendenza condizionale anche se nella realtà non è vera.

Nelle **reti bayesiane**, si **tiene conto delle dipendenze tra variabili**, infatti sono modelli che utilizzano DAG (grafici direzionati aciclici) e il likelihood è calcolato come prodotto di probabilità condizionate, sulla base delle dipendenze espresse.



Ci sono altri classificatori generativi, come: QDA, LDA, GNB e MNB, i quali fanno diverse assunzioni e sono più complessi.

Classificatori Discriminativi

Un altro tipo di classificatori, che non studiano lo storico e la probabilità di una certa classe bensì si cerca di **predire la classe direttamente dai dati osservati**, sono i **classificatori discriminativi**.

Essi tracciano i confini tra una classe ed un'altra, infatti cercano una **regola che separi le classi**.

I vantaggi dei modelli discriminativi rispetto ai bayessiani, sono che:

- Predicono con maggior accuratezza;
- Sono robusti in presenza di errori nel training set;
- Facile calcolo per una nuova classificazione.

Gli svantaggi sono legati al tempo di addestramento e alla poca interpretabilità dei pesi di una funzione.

I classificatori discriminativi costruiscono una funzione di decisione sui dati osservati, con i pesi calcolati per ogni attributo.

La **classificazione** può **essere lineare o non lineare e binaria o non binaria**(4 tipi).

Il **perceptron** è un **algoritmo di classificazione binaria lineare** e adotta una funzione predittiva lineare ottenuta combinando i pesi con il vettore degli attributi.

Quindi si ha:

$$f(X) \begin{cases} 1 & \text{se } \vec{w} \cdot \vec{x} + b > \theta \\ 0 & \text{altrimenti} \end{cases}$$

dove:

- $\vec{w}$ è il vettore dei pesi(solitamente i pesi iniziali sono tutti uguali a 0 o valore random = 0);
- $\vec{x}$ è il vettore degli attributi(tupla);
- b è la costante di bias.

Sia poi:

- r è il **learning rate**(tra 0 e 1) un iper-parametro che serve per l'aggiornamento dei pesi quando il classificatore commette un errore;
- y_j' è la classe predetta dal perceptron per la j -esima tupla del training set;
- $y_j' - y_j$ è l'errore generato dal perceptron (valore compreso tra 0 e 1).

Quando si fa un errore di predizione avremmo che:

$$w_{j+1}(t) = w_j(t) + r(y_j'(t) - y_j)x_{j,i}$$

t è il contatore di aggiornamento dei pesi.

Il perceptron solitamente effettua un **apprendimento online**, ossia che gli elementi di training vengono presi in considerazioni uno alla volta e i pesi sono aggiornati ogni volta che ogni elemento è preso in esame.

Ciò significa anche che i primi dati che dovranno essere classificati, verranno classificati con un modello poco allenato -> predizioni iniziali spesso sbagliate.

Possiamo anche utilizzare il **learning offline(batch)**, ossia si predicono tutti i dati e si calcola l'errore medio, alla fine di un'epoca(dopo che tutto il dataset è stato predetto). Dall'errore medio si aggiornano i pesi e appena che l'errore medio scende al di sotto di una certa soglia γ l'algoritmo termina.

La soglia per determinare se un elemento appartiene o non appartiene ad una classe può essere fissata, oppure applicata includendola nella funzione predittiva come un peso aggiuntivo. Si introduce quindi una nuova variabile dummy $x_{j,k+1} = -1$.

$\vec{w} \cdot \vec{x} + b > \theta$ lo possiamo spostare a sinistra e si ha $\vec{w} \cdot \vec{x} + b - \theta > 0$.

La predizione della classe diventa:

$$y_j'(t) = w_0(t)b + w_{j,1}(t)x_{j,1} + \dots + w_{j,k}(t)x_{j,k} + w_{j,k+1}(t)x_{j,k+1}$$

L'aggiornamento avviene come nel resto dei pesi.

Il perceptron può essere esteso alla classificazione con più classi. Gli approcci che si possono seguire sono:

- **One Vs One (OVO):** date $C_1, C_2, \dots, C_n$ le classi del dataset si fissa una soglia θ . Si allena il modello a classificare binariamente le tuple di classe C_i e C_j , l'output sarà $O_{i,j}$ che sarà uguale a $-O_{j,i}$. Si assegna alla tupla la classe x C_i per cui ha valore massimo $\sum_{j=1, i \neq j}^k O_{i,j}$. Se il valore tende a 1 si assegna la classe i , se il valore tende a -1 la classe j .
- **One Vs All (OVA):** date $C_1, C_2, \dots, C_n$ le classi del dataset si fissa una soglia θ , x si assegna alla classe i se O_i ha valore massimo.

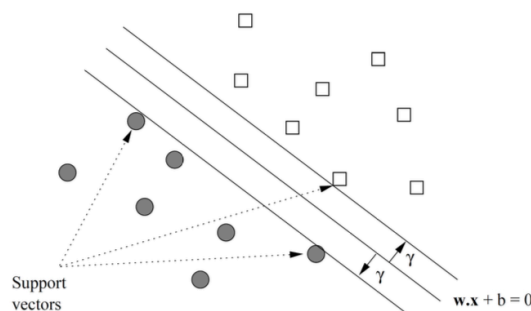
Il perceptron trova solo funzioni lineari, quindi lavora con dati linearmente separabili, se non sono così occorre fare una trasformazione. Infatti, se i dati non sono linearmente separabili si opta per trasformare i dati in un altro spazio e separarli lì.

Un altro classificatore importante è il **Support Vector Machines (SVM)**, che è un classificatore binario basato su un iperpiano che separa al meglio i dati.

Gli elementi che determinano l'iperpiano ottimale sono:

- **I vettori di supporto:** ossia i punti di frontiera tra due classi;
- **I margini:** ossia le distanze di tali vettori dall'iperpiano.

Se i dati sono linearmente separabili, esistono infiniti piani di separazione, per questo motivo si sceglie quello che minimizza l'errore di classificazione, ergo che massimizza il margine tra i dati di due classi.



L'equazione dell'iperpiano separatore H sarà $y = w \cdot x + b = 0$ dove:

- x è il vettore dei punti di una tupla;
- w è il vettore dei pesi;
- b la costante di bias;
- γ si indica il **margine**, ossia il valore che si deve massimizzare attraverso w e b ;

Possiamo quindi indicare i due iperpiani che passano per i vettori di supporto delle due classi come segue:

- $H_1: \mathbf{w} \cdot \mathbf{x} + b = -\gamma$ i punti di classe -1 stanno al di sotto di H_1 ;
- $H_2: \mathbf{w} \cdot \mathbf{x} + b = \gamma$ i punti al di sopra di 1 stanno sopra H_2 .

Si ricercano sempre dei coefficienti w e b che massimizzano il margine, però se moltiplichiamo essi per qualsiasi costante avremmo la stessa linea ma con margine più grande.

Qual è quindi la linea con il margine vero? Si deve fare la normalizzazione dei pesi per saperlo. In sostanza per una matematica oscura e sconosciuta, dobbiamo trovare il valore di $\|\mathbf{w}\|$ minimo. Perché?

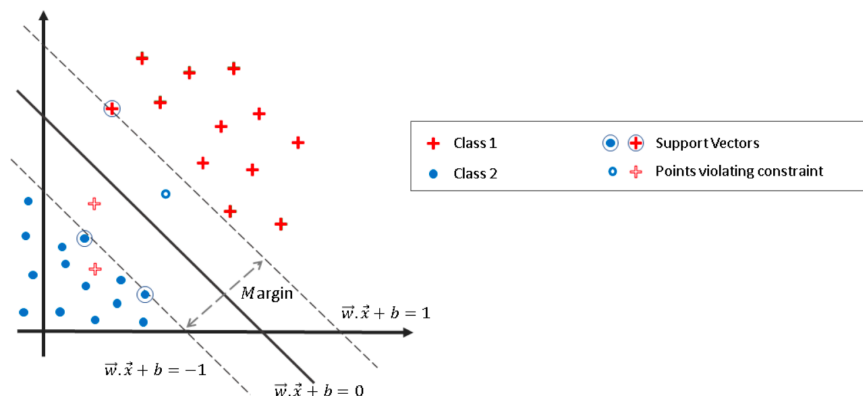
$$\gamma = \frac{1}{\|\mathbf{w}\|}$$

Ricordando che $\|\mathbf{w}\| = \sqrt{w_1^2 + \dots + w_n^2}$.

Più piccolo è $\|\mathbf{w}\|$ più grande sarà il valore di γ . Per trovarlo si applica la discesa del gradiente, però in pratica si cerca $\min \|\mathbf{w}^2\|$ che semplifica i calcoli.

La classificazione descritta finora è detta **“hard margin”** poiché nessun punto può trovarsi nella **regione compresa tra H_1 e H_2** . Questa classificazione è molto sensibile al rumore, dato che se un punto di una classe si trova molto vicino all'altra classe, si deve spostare l'iperpiano e ridurre il margine.

Una variante è la classificazione **“soft margin”** che non è sensibile al rumore, dato che nella zona di margine sono ammessi gli outlier.



Con l'hard margin, rischiamo di incorrere in overfitting se i dati non sono perfettamente separabili, invece con il soft margin si può ridurre l'accuratezza.

Per questo motivo bisogna trovare un trade-off tra la larghezza del margine e il numero di violazioni. Si introducono n variabili $\epsilon_1, \epsilon_2, \dots, \epsilon_n$ tale che:

- Se $\epsilon_i = 0$ allora il punto ricade al di fuori della zona di margine;
- Se $0 < \epsilon_i < 1$ allora il punto ricade all'interno del margine;
- Se $\epsilon_i \geq 1$ allora il punto si trova oltre l'iperpiano separatore.

Quindi per trovare dei pesi minimi che rendono l'errore trascurabile si procede alla seguente maniera:

$$\begin{cases} \min(\frac{1}{N} \sum_{i=1}^n \epsilon_i + \lambda \|\vec{w}\|^2) \\ \epsilon_i \geq 0 \end{cases}$$

Dove λ è l'**iper-parametro** che costituisce il **trade-off tra larghezza del margine e numero di violazioni tollerate**.

Se i dati non possono essere separati in modo pulito attraverso una semplice linea retta, allora possiamo **applicare il mapping**, proiettando in uno spazio più grande i punti.

Questo si fa attraverso **funzioni di mapping ϕ** , e i dati che prima erano mescolati diventano poi linearmente separabili. A quel punto si può applicare semplicemente la linea di separazione.

SVM è quindi un classificatore binario, ma può essere applicato anche a problemi multi-classe, utilizzando gli approcci descritti precedentemente come One-Vs-One o One-vs-All. Si presta bene a classificare anche dati ad alta dimensionalità(tante feature) e la complessità nell'addestramento è data solo dal numero di tuple.

Apprendimento Lazy

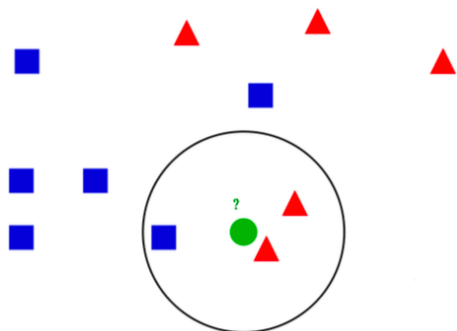
L'apprendimento descritto finora è un **apprendimento Eager**, ossia un supervised learning, in cui il sistema cerca di costruire una **descrizione completa e generalizzata del modello a partire dai dati di training**.

Un learning totalmente diverso è l'apprendimento lazy, ossia un tipo di apprendimento dove si memorizza un training set e calcola la funzione di classificazione quando bisogna classificare un nuovo dato.

Il tempo impiegato per la predizione richiede più tempo, dato che non è presente un modello che ha già una formula, come quella di regressione lineare che applicata ai nostri input ritorna un valore.

La funzione di predizione è approssimata localmente(si adatta alla distribuzione locale dei dati) e viene utilizzata per grandi dataset con pochi attributi, dato che come nel k-nn si utilizza la dimensionalità(perde di valore all'aumentare della dimensionalità).

Il K-NN o K-Nearest Neighbors, per stimare la classe di una tupla, si prendono in esame le k tuple più vicine(le più simili) alla tupla X. Si assegna a X la classe più ricorrente tra le classi delle k tuple scelte.



Come descritto prima si calcola la funzione di appartenenza sul nuovo dato, e non se ne applica una pregressa.

In questo caso vengono presi $k=3$ punti più vicini e attraverso la moda sappiamo quale classe assegnare.

Potremmo avere anche delle varianti dove:

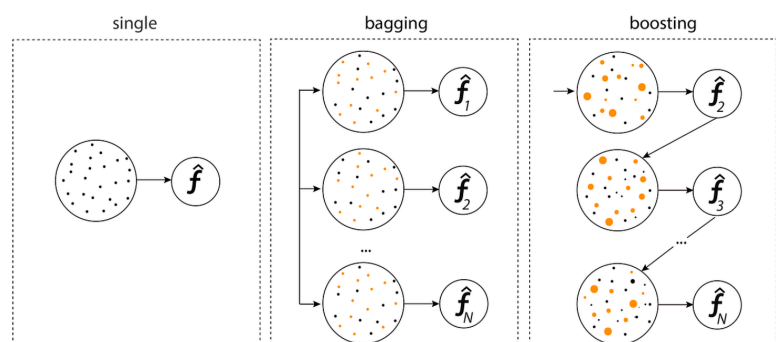
- si **pesa la distanza**, l'elemento più vicino ha un peso maggiore e si classifica sulla base della distanza.
- per **valori continui**, si può classificare attraverso la media dei valori degli attributi delle tuple vicine.

Apprendimento Ensemble

L'**ensemble learning** combina due o più modelli, dello stesso tipo solitamente, per ottenere migliori performance di predizione. Dato che richiede molta computazione è utilizzato per combinare algoritmi di apprendimento veloci, anche se non hanno un'elevata accuratezza.

Il **boosting** costruisce **modelli in sequenza**, non in parallelo, dove ogni classificatore viene addestrato **dando più peso ai punti su cui i modelli precedenti hanno sbagliato**. Così facendo l'errore totale si riduce, infatti, modelli semplici diventano molto forti dopo che sono stati combinati.

Il **bagging** addestra in parallelo i classificatori e combina le predizioni dei classificatori per ottenere la predizione finale.



Si ricorda che l'addestramento può essere fatto sul **training set** o su un **set di campioni ottenuti mediante bootstrap sampling**.

Bagging

Si hanno $M_1, M_2, \dots, M_k$ i modelli da combinare. Si addestrano sul training set o sul campione di dati e infine a seconda dell'obiettivo del modello:

- se l'obiettivo è **predire** i dati, si restituisce la **media** dei valori predetti;
- se l'obiettivo è la **classificazione** allora si restituisce la **classe più frequente**.

Un esempio è il random forest che combina i risultati dei diversi alberi decisionali. Ogni albero è addestrato su un campione random e si splittano gli attributi nei 3 alberi. Solitamente lo split è:

- $m = \sqrt{p}$ per la classificazione;
- $p/3$ per la predizione.

Infine, il valore restituito è condizionato dall'obiettivo perseguito.

Boosting

Siano $M_1, \dots, M_k$ modelli e ad ogni tupla X si associa un peso w_x , uguale per tutte.

Si addestra il classificatore M_i sul training e nel prossimo classificatore le tuple che hanno maggior error rate si aumenta il peso e viceversa. Il classificatore M_{i+1} si concentrerà sugli errori fatti dal classificatore precedente. Infine, tutti i classificatori vengono combinati linearmente per produrre la predizione finale.

Uno degli algoritmi boosting più popolari e importanti è l'algoritmo **Adaboost(Adaptive boosting)**.

I classificatori deboli utilizzati sono gli **alberi decisionali con due foglie(stump)** un test su una sola variabile), che combinati danno alla luce un classificatore veloce e accurato.

Ogni tupla X del dataset ha un peso $w(X) = 1/N$, dove N sono le tuple del dataset. Si crea lo stump D_i splittando sull'attributo A che minimizza il gini index. Si calcola un peso legato all'accuratezza e si aumentano i pesi legati alle tuple classificate erroneamente e viceversa.

Infine, si combinano i risultati degli stump, usando i pesi gli opportuni pesi. Per calcolare il peso dello stump, si fa attraverso totalerror, ossia le tuple classificate erroneamente di D_i .

$$\text{Il peso di } D_i \text{ è dato da: } \frac{1}{2} \log\left(\frac{1 - \text{totalerror}}{\text{totalerror}}\right).$$

Quando una tupla è classificata erroneamente, l'aggiornamento del peso è svolto nella seguente maniera:

$$w^{i+1}(X) = w^i(X) \cdot x \cdot e^{w(D_i)}$$

Per diminuire il peso, si utilizza la seguente formula:

$$w^{i+1}(X) = w^i(X) \cdot x \cdot e^{-w(D_i)}$$

Il bagging è più adatto per ridurre l'overfitting, invece, il boosting è più adatto a combinare classificatori scadenti.

Validazione di classificazione

Le misure per validare il nostro classificatore sono svariate e si applicano a seconda della tipologia del classificatore stesso.

Per rappresentare l'accuratezza della classificazione di classificatori binari o multiclasse si utilizza la **matrice di confusione**. Ogni **riga contiene i valori reali**, ogni **colonna i valori predetti**, e dalla fusione dei valori reali con quelli predetti si ottiene la seguente matrice:

		Predetti			Somma
		Gatto	Cane	Coniglio	
Reali	Gatto	5	2	0	7
	Cane	3	3	2	8
	Coniglio	0	1	11	12
Somma		8	6	13	27

Il valore somma alla fine di ogni riga indica il numero di valori reali appartenenti ad una determinata classe.

Il valore somma alla fine di ogni colonna indica il numero di valori predetti appartenenti ad una classe.

Tutti gli elementi appartenenti alla diagonale sono i valori in cui il classificatore ha classificato vera una classe.

L'accuratezza di un classificatore M indicata con **acc(M)**

misura la percentuale di tuple classificate correttamente dal classificatore.

$$\text{acc}(M) = \frac{\text{somma diagonale principale (tranne ultimo elemento)}}{\text{ultimo elemento delle somme}} = \frac{5 + 3 + 11}{27} = \frac{19}{27}$$

Possiamo quindi anche calcolare l'**error rate**:

$$\text{ErrorRate: } 1 - \text{acc}(M) = 1 - \frac{19}{27} = \frac{8}{27}$$

Nel caso di due sole classi con: P(positiva) e N(negativa) possiamo definire:

- **True positive(Tpos)**: tuple di classe P che sono state classificate correttamente;
- **True Negative(Tneg)**: tuple di classe N che sono state classificate correttamente;
- **False positive(Fpos)**: tuple di classe N che sono state classificate come P;
- **False Negative(Fneg)**: tuple di classe P che sono state classificate come N;

Le misure di accuratezza con due classi sono:

- **Recall(TPR)->** quanto il modello classifica bene gli elementi appartenenti alla classe Positiva:
 - $\frac{|Tpos|}{|Pos|}$
- **Specificità(TNR)->** quanto il modello classifica bene gli elementi appartenenti alla classe Negativa:
 - $\frac{|Tneg|}{|Neg|}$
- **False Positive Rate(FPR)->** quanto il modello ha classificato erroneamente elementi positivi che dovevano essere negativi:
 - $\frac{|Fpos|}{|Fpos+Tneg|}$
- **False Discovery Rate(FDR)->** quanto il modello ha sbagliato a classificare gli elementi positivi:
 - $\frac{|Fpos|}{|Fpos+TPos|}$
- **Precision(prec(M)) ->** quanto il modello ha classificato correttamente elementi positivi:
 - $\frac{|Tpos|}{|Fpos+TPos|}$
- **Accuratezza(acc(M))->** quanto il modello ha correttamente predetto i valori:
 - $\frac{|Tpos|+|Tneg|}{|Neg|+|Pos|}$
- **F1 score->** metrica che indica la capacità del modello di non fare errori(precisione) e di trovare tutti i positivi(recall):
 - $2 \times \frac{\text{Prec}(M) \times \text{Rec}(M)}{\text{Prec}(M) + \text{Rec}(M)}$ media armonica tra precisione e recall. Si manipola per bilanciare i due valori

Spesso la distinzione tra due classi avviene attraverso un valore **soglia σ** . Le performance del classificatore sono valutate al variare di σ tramite **curva ROC**.

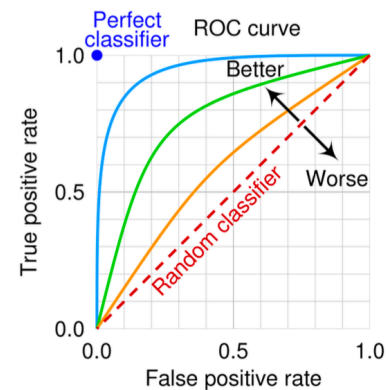
La **curva ROC** rappresenta il **True Positive Rate (TPR)** in funzione del **False Positive Rate**, al variare di σ .

Se la soglia è alta, tutte le tuple sono classificate come positive avendo un FPR e un TPR pari a 0.

Se la soglia è molto bassa, allora tutte le tuple saranno classificate come positive e quindi il valore di FPR e TPR sarà pari a 1.

Il classificatore che ha $TPR = 0.5$ e $FPR = 0.5$ è un classificatore random, e non è un buon classificatore. I classificatori la cui curva ROC è al di sotto del classificatore random, sono errati.

Il classificatore ideale presenta una curva ROC, il cui TPR aumenta velocemente a 1 e l'FPR si mantiene pari a 0.



La curva ROC si utilizza nel caso di dataset bilanciati. In caso contrario, quindi in dataset sbilanciati, si utilizza la **curva di precision-recall** che rappresenta la **precisione in funzione della recall al variare di σ** . Come **indicatore dell'accuratezza** del classificatore si fa riferimento all'**area sotto la curva ROC o AUC**, che assume valori tra 0 e 1. Maggiore è il valore e maggiore è l'accuratezza.

Per validare un classificatore si considerano diversi partizionamenti dei dati in training set e test set. Due metodi di validazione sono:

- **Holdout**-> si fissa una percentuale X e si partiziona il dataset in due set indipendenti, ossia il training set ($X\%$ del dataset) e ($100-X\%$ del dataset). Si addestra il modello sui dati di training e attraverso il test set si misura l'accuratezza.
- **K-Fold cross validation**-> si partiziona il dataset in k partizioni $D_1, \dots, D_k$ della stessa dimensione. Si addestra il modello sulle dataset ottenuto dall'unione delle partizioni $D_1, \dots, D_{i-1}, D_{i+1}$ e infine si testa sulla partizione D_i .

Regressione

La **regressione** è la tecnica di predizione più importante e stima la **relazione tra una variabile dipendente** e una o più **variabili indipendenti** chiamate predittive o **predittori**.

La regressione può essere di svariati tipi:

- **Regressione lineare semplice**-> modella la relazione tra una variabile di risposta e un singolo predittore:
 - $y = w_0 + w_1x$
dove w_0 è l'**intercetta** e w_1 è la **pendenza** e sono i **coefficienti di regressione**.
- **Regressione lineare multipla**-> modella la relazione tra una variabile e più predittori:
 - $y = w_0 + w_1x_1 + w_2x_2$
- **Regressione non lineare** -> la funzione non è lineare, però in alcuni casi si può trasformare in una lineare:
 - $y = w_0 + w_1x + w_2x^2 + w_3x^3$ si può trasformare in $y = w_0 + w_1x + w_2x_2 + w_3x_3$,
con $x_2 = x^2, x_3 = x^3$
- **Regressione logistica**-> per predire un valore attraverso una combinazione lineare delle variabili predittive. Alla base vi è la funzione logistica che è la distribuzione di probabilità. Il modello che calcola la probabilità avrà al suo interno la seguente funzione:

- $$P(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Grafi

Un **sistema complesso** è un insieme caratterizzato da **entità che interagiscono**, la cui **complessità** deriva sia dalle loro **interazioni** e la difficoltà nel comprenderle e sia dalla **dimensione** del sistema stesso.

Le **reti(o grafi)** sono il modello matematico che **permette di codificare le interazioni tra componenti** di un sistema complesso.

Si ricordano le nozioni sui grafi acquisite durante il corso di strutture discrete, ossia:

Un grafo è una coppia $G(V,E)$, dove:

- V è l'insieme dei vertici -> componenti del sistema;
- E è l'insieme degli archi -> interazione tra una coppia di vertici;

Un esempio di rete è una rete di comunicazione: ogni router rappresenta un vertice del grafo, mentre i collegamenti fisici o wireless con altri dispositivi costituiscono gli archi della rete.

La scienza che si occupa di analizzare le reti è la Network Science, e l'obiettivo è la comprensione di regole fondamentali o principi organizzativi comuni che vengono seguiti dalle reti reali.

Un **grafo è indiretto(non orientato)** se presi $\forall(a,b) \in E$ allora **esiste** anche $(b,a) \in E$. Questo implica che l'**interazione tra due nodi sia reciproca**: ad esempio, nelle reti sociali l'interazione tra due persone è bidirezionale.

Un **grafo è diretto(orientato)** se presi $\forall(a,b) \in E$ non è garantito che esista $(b,a) \in E$. Solitamente l'**interazione è unidirezionale**: ad esempio, nei canali broadcast un solo nodo interagisce unidirezionalmente con tutto il resto della rete.

$$E_{non\ orientato} = \{(a,b), (b,a), (b,d), (d,b), (d,c), (c,d), (c,b), (b,c)\}$$

$$E_{orientato} = \{(a,b), (d,b), (c,d), (b,c)\}$$



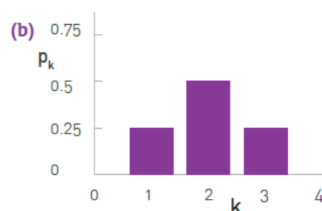
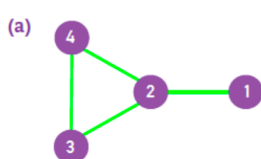
Questa distinzione è importante al fine del calcolo del **grado di un nodo n**. Il grado di un nodo è il **numero di nodi adiacenti a n** nel grafo, se il grado è 0 il nodo è isolato.

In un grafo diretto si distingue:

- **Grado uscente del nodo n**: numero di nodi con cui interagisce il nodo;
- **Grado entrante del nodo n**: numero di nodi che interagiscono con il nodo preso in esame;
- **Grado totale del nodo n**: grado uscente di n + grado entrante del nodo n.

Il **grado medio della rete** si calcola con il seguente rapporto: $\frac{|E|}{|V|}$.

Possiamo quindi analizzare la **distribuzione dei gradi P**, ossia la distribuzione di probabilità, dove p_k rappresenta la probabilità che un **nodo random abbia grado k**. P_k si ottiene dividendo N_k di nodi con grado k per il numero di nodi della rete.



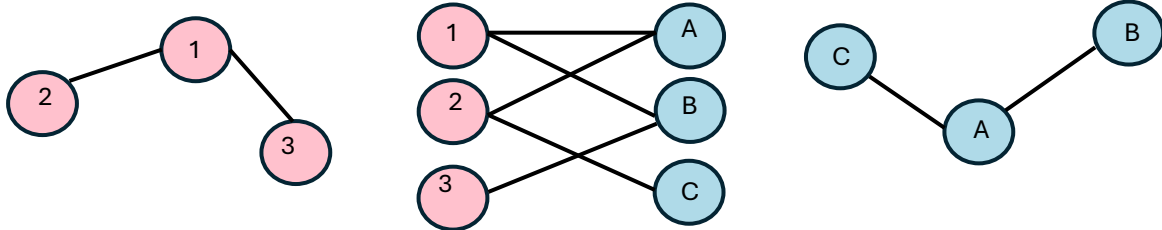
$$P_1 = \frac{1}{4} = 0.25 \text{ (solo il nodo 1 ha grado 1)}$$

$$P_2 = \frac{2}{4} = 0.5 \text{ (i nodi 3 e 4 hanno grado 2)}$$

$$P_3 = \frac{1}{4} = 0.25 \text{ (solo il nodo 2 ha grado 3)}$$

Un **grafo bipartito** è un grafo contenente due insiemi disgiunti di nodi, dove non esistono archi che collegano nodi dello stesso insieme, ma esistono solo archi che collegano elementi di U a elementi di V. Il caso si può estendere al caso n di insiemi.

Da qui possiamo ricavare le **proiezioni rispetto a U e rispetto a V**, ossia grafi i cui **nodi dello stesso insieme sono connessi** se hanno almeno un **nodo adiacente comune nel grafo bipartito**.



Studiando sempre le iterazioni tra nodi di un grafo, possiamo dire che:

- Un **grafo è completo**, quando **tutte le coppie** del grafo sono **collegate** da un arco;
- Un **grafo è regolare**, quando **tutti i nodi** del grafo hanno lo **stesso grado**.

Tra due nodi di un grafo u e v possiamo cercare una **sequenza ordinata di archi che li collega**. Il **cammino** che si cerca ha i nodi $i_0 = u$ e $i_n = v$, dove la quantità n è la lunghezza del cammino. Possono esistere più cammini tra due nodi.

Un cammino che ha $i_0 = i_n$, ossia che il **nodo di partenza e nodo finale coincidono** è detto **ciclo**. Un ciclo di lunghezza 1 è detto **cappio**, invece un grafo che non ha cicli è detto **aciclico**.

Partendo sempre dalla definizione di cammino, possiamo dire che due nodi si dicono connessi se esiste un cammino che li collega. Se **tutti i nodi sono connessi**, si parla di **grafo connesso**.

Un grafo disconnesso è formato dall'unione di due o più sotto-grafi connessi, chiamati **componenti connesse**.

Infine, se il grafo è orientato:

- È **fortemente connesso** se presi tutti i nodi sono connessi;
- È **debolmente connesso** se è connesso solo attraversi archi non direzionati (togliamo la direzionalità agli archi e verifichiamo);

Il diametro è la distanza massima tra due nodi (i più lontani) del grafo.

I **grafi** possono essere **pesati**, ossia ad ogni nodo possiamo **associare dei valori numerici**, oppure **etichettati**, ovvero che ad ogni arco **associamo etichette o stringhe**.

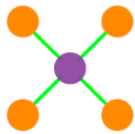
Clustering

Il **coefficiente di clustering** C_n di un nodo n misura **quanto i suoi vicini siano connessi tra loro**. È un **indicatore della densità locale** della rete attorno al nodo

$$C_n = \frac{2L_n}{K_n(K_n - 1)}$$

Dove:

- K_n è il grado del nodo n ;
- L_n il numero di archi esistenti tra i K_n adiacenti;
- C_n è un valore compreso tra 0 e 1.



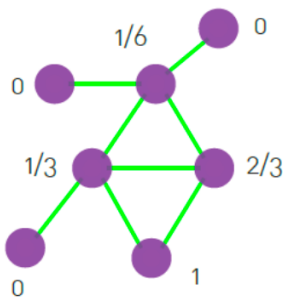
In questo caso gli archi esistenti tra i K_n nodi adiacenti è uguale a 0. Infatti, non si contano gli archi incidenti al nodo preso in considerazione, bensì quelli dei nodi adiacenti.

Quindi il coefficiente di clustering risulterà 0 ($C_n = \frac{2 \cdot 0}{(4 \cdot (4-1))} = \frac{0}{(4 \cdot 3)} = \frac{0}{12} = 0$).

Il **coefficiente di clustering medio** è la media dei coefficienti di clustering sui nodi della rete:

$$\langle c \rangle = \frac{1}{N} \sum_{i=1}^n c_i$$

Il **coefficiente di clustering globale** misura in media **quanto è probabile che due nodi hanno un vicino in comune**. Per calcolare il valore si esegue il rapporto tra il numero di triangoli (preso 3 volte dato che il triangolo contiene tre triple) e il numero di triple di nodi connessi tra loro (anche cammini di 3 nodi).



$$C_{\Delta} = \frac{3 \cdot \text{triangoli}}{\text{triplette connesse}}$$

Misure di centralità

La **centralità** di un nodo è un insieme di misure, che servono a **quantificare quanto un nodo è importante** all'interno di una rete. L'idea di importanza può essere definita in diversi modi, per questo esistono tipi diversi di centralità che catturano aspetti diversi.

Degree centrality

La **degree centrality** pone l'**importanza sul grado** di un nodo. Infatti, misura quanto un nodo è direttamente connesso con gli altri nodi. Più alto è il grado e più importante è il nodo.

Betweenness centrality

La **betweenness centrality** quante volte un **nodo si trova sul percorso tra altre coppie** di nodi. Se esso è presente in molti cammini, allora è importante dato che agisce come ponte. Se il nodo in questione viene rimosso allora la comunicazione si interrompe.

Closeness centrality

La **closeness centrality** misura quanto un **nodo è vicino a tutti gli altri nodi** della rete. Misura quindi la velocità media con cui un'informazione, partendo da v, può raggiungere gli altri nodi del grafo.

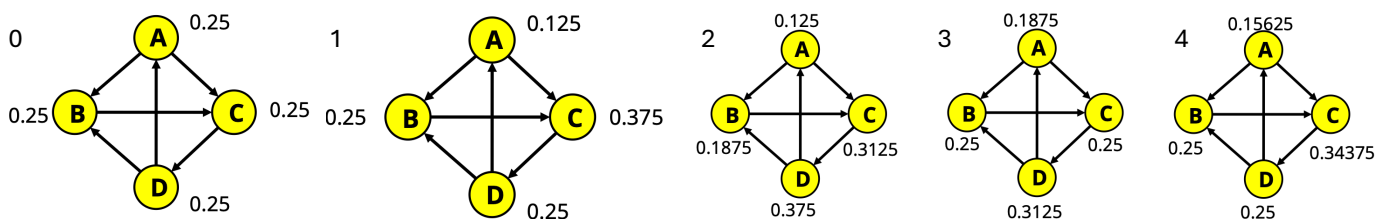
Pagerank centrality

La **pagerank centrality** si basa sull'osservazione che le connessioni di un nodo con gli altri nodi non hanno tutte la stessa importanza. Infatti, **connessioni a nodi con grado elevato hanno un peso maggiore** e viceversa.

Un nodo è importante se è connesso ad altri nodi importanti nella rete.

Per verificare l'importanza attraverso le connessioni a nodi importanti, abbiamo che:

- In una rete diretta si ha a disposizione uno score iniziale(importanza), uguale per tutti.
- Il complessivo è pari a 1, quindi lo score per ogni nodo sarà $1/n$, con n numero di nodi.
- Ogni nodo dovrà poi cedere il proprio score in parti uguali tra i suoi adiacenti.
- Si reitera il passaggio di distribuzione dello score finché non si raggiungono valori stabili.



0) SITUAZIONE INIZIALE -> PRIMA ITERAZIONE

- A divide il proprio score per cederlo a B(0.125) e C(0.125)
- C cede interamente il proprio score a D(0.25)
- D divide il proprio score per cederlo a B(0.125) e A(0.125)
- B cede il proprio score interamente a C(0.25)
- INFINE, CALCOLIAMO B=(0.125+0.125), D=(0.25), C=(0.125+0.25), A=(0.125)

1) PRIMA ITERAZIONE -> SECONDA ITERAZIONE (si ripete)

- A divide il proprio score per cederlo a B(0.0625) e C(0.0625)
- C cede interamente il proprio score a D(0.375)
- D divide il proprio score per cederlo a B(0.125) e A(0.125)
- B cede il proprio score interamente a C(0.25)
- INFINE, CALCOLIAMO B=(0.0625+0.125), D=(0.375), C=(0.0625+0.25), A=(0.125)

Subgraph matching

L'**isomorfismo tra grafi** (graph matching), consiste nel verificare se **due grafi sono uguali** tra loro. Due grafi sono isomorfi quando sono lo stesso grafico, ma **disegnati diversamente** e/o con **nomi diversi ai nodi**.

Formalmente:

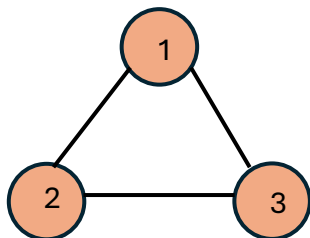
$G_1 = (V_1, E_1)$ $G_2 = (V_2, E_2)$ sono **isomorfi** se esiste una funzione biiettiva **se** $f: V_1 \rightarrow V_2$, detta **mapping**. Quindi, $(u, v) \in E_1$ se e solo se $(f(u), f(v)) \in E_2$.



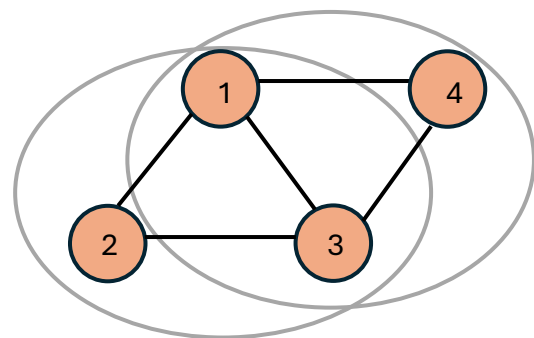
L'isomorfismo tra un grafo G e sé stesso è detto **automorfismo** di G . Un grafo può avere anche più di un automorfismo.

Il **subgraph matching** (**sottografo isomorfismo**) consiste nel verificare se un grafo (**grafo query**) è contenuto in un grafo più grande (**grafo target**).

In questo caso la **funzione** è solo **iniettiva** (un elemento di A è associato ad un e un solo elemento di B), perché possono esserci nodi in più nel grafo target che non vengono utilizzati. L'importante è che siano mappati tutti i nodi del grafo query.



GRAFO QUERY



GRAFO TARGET

In generale la soluzione non è unica, infatti, nell'esempio soprastante ci sono due matching possibili.

Il graph matching è un problema NP-hard, però non si sa se è completo, invece, il subgraph matching è un problema NP-completo.

Gli algoritmi che vengono utilizzati per la risoluzione dei problemi, si basano sulla trasformazione del grafo in una **forma canonica**.

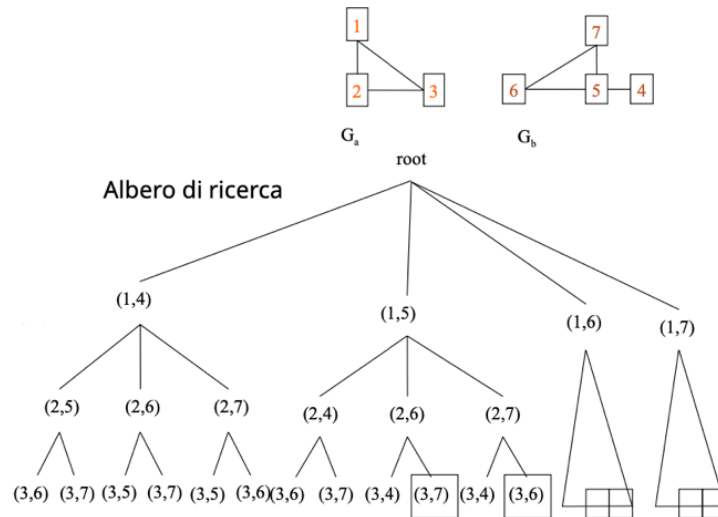
Si prendono i due grafi G_1 e G_2 e si convertono in una rappresentazione univoca ad esempio: stringhe, liste o altre varianti implementate a seconda degli algoritmi.

Se le forme canoniche sono uguali ($\text{Canon}(G_1) = \text{Canon}(G_2)$) allora i due grafi sono isomorfi.

Una delle possibili soluzioni è anche l'approccio più semplice da seguire (il meno efficiente) per risolvere il subgraph matching consiste nel provare **tutte le possibili associazioni tra i nodi** del grafo query e i nodi del grafo target. Questa soluzione è chiamata **soluzione brute-force**, e le esplorazioni di tutti i possibili mapping è possibile attraverso un **albero di ricerca**.

Ogni cammino dalla radice ad una delle foglie rappresenta un possibile mapping. L'albero è così organizzato:

- la radice rappresenta lo stato iniziale (nodi ancora non mappati);
- ogni livello corrisponde alla scelta del nodo target;
- ogni arco dell'albero rappresenta una mappatura;
- ogni cammino dalla radice alla foglia, rappresenta un possibile mapping.



L'algoritmo termina quando:

- tutti i nodi del grafo query sono stati mappati;
- tutte le alternative sono state esplorate.

Se il grafo ha n nodi, esistono $n!$ possibili modi per mappare i nodi, per questo motivo questo algoritmo diventa subito ingestibile. Ad esempio, con $n = 5$ nodi abbiamo $n! = 120$ possibili modi per mappare i nodi.

A causa di questa ingestibilità, sono utili strategie di ricerca per velocizzare la computazione, ossia:

- **look-ahead:** predire anticipatamente che il matching parziale non porterà ad alcuna soluzione finale; potrebbe quindi essere utile osservare:
 - se il nodo target non ha abbastanza vicini per soddisfare le connessioni del nodo query;
 - alcuni archi del query non possono essere soddisfatti nel target.
(PREVISIONE)
- **backtracking:** abbandonare soluzioni parziali quando ci accorgiamo che non portano a soluzioni finali. Si torna quindi indietro allo step precedente.
(IN CORSO D'OPERA)

L'algoritmo di Ullman utilizza il **grado del nodo** per fare look-ahead e backtracking filtrando lo spazio delle possibili soluzioni. Se il **nodo della query ha un grado maggiore del nodo** corrispondente del grafo **target**, allora si effettua **pruning**.

In caso contrario possiamo continuare a visitare il ramo, dato che il nodo del grafo target può avere più nodi vicini e connessi (si sta cercando un sottografo del target).

L'algoritmo VF (Vento-Foggia) usa la state **space representation (SSR)**, ossia che ogni **stato è un mapping parziale $M(s)$** e quando si aggiunge una nuova coppia si produce un nuovo stato. Quando si arriva alle foglie si ha il mapping completo. Si aggiunge una nuova coppia secondo le **regole di fattibilità**.

Le regole di fattibilità designano degli insiemi di possibili nodi terminali (vicino ai nodi adiacenti), che scartano subito rami impossibili.

Gli insiemi definiti dalle regole di fattibilità dell'algoritmo vento-foggia sono:

- $Match_{Q/T}(s)$ = insieme dei **nodi query/target già mappati** allo stato s ;
- $Term_{Q/T}(s)$ = insieme dei **nodi query/target terminali**, ossia adiacenti ai nodi query/target già mappati;
- $Rem_{Q/T}(s)$ = **restanti nodi** del grafo query/target allo stato s .

Gli insieme ovviamente sono separati e sono stati raggruppati solo per evitare ripetizioni. L'**insieme $P(s)$ dei probabili candidati** è dato dalle **coppie (n,m)** , dove $n \in Term_Q(s)$ e $m \in Term_T(s)$.

Si ricorda che durante la **ricerca** all'interno dell'albero si procede **per profondità** (depth-first).

Le regole prevedono che una coppia $(p,q) \in P(s)$ è aggiunta al mapping parziale $M(s)$ se e solo se:

- 1) Se si aggiunge una nuova coppia al mapping, tutti gli archi già mappati devono rimanere coerenti -> **consistenza del nuovo stato**;
- 2) Il numero di nodi terminali del grafo query connessi a q deve essere inferiore o uguale al numero di nodi terminali del grafo target connessi a t -> si evita di scendere in alberi morti -> **look-ahead a 1 livello**;
- 3) Il numero di nodi rimanenti (rem_Q) del grafo query connessi a q deve essere inferiore o uguale al numero di nodi rimanenti (rem_T) del grafo target connessi a t -> controllo anticipato più largo -> **look-ahead a 2 livelli 1**;

Per grafi diretti o orientati, gli insiemi citati prima si sdoppiano per nodi successori/predecessori. Le regole, quindi, verranno modellate sulla base dell'orientazione.

La complessità computazionale sarà uguale:

- Caso migliore: ad ogni passo c'è un solo candidato che soddisfa le regole di fattibilità -> $O(N^2)$
- Caso peggiore: si deve esplorare l'intero albero di ricerca -> $O(N!N)$

La complessità spaziale sarà di circa $O(N^2)$. VF2 ottimizza lo spazio a $O(N)$.

Un approccio alternativo alle regole di fattibilità, che riducono lo spazio di ricerca ma sono costose da implementare, è l'**algoritmo RI**.

Quest'**algoritmo si concentra sull'ordine in cui i nodi della query vengono processati** nell'albero di ricerca. Un ordine efficace dei nodi velocizza il matching, e l'ordinamento è calcolato indipendentemente dal grafo target (static ordering) ma in altri è fatto anche sulla base del grafo citato.

Prima di esplorare l'albero di ricerca, assegniamo una priorità ai nodi del grafo query basata su criteri quali: grado, vincoli e nodi che massimizzano il pruning.

Si ordinano in modo di **massimizzare la probabilità che un cammino venga tagliato il prima possibile** e poi si mappano i nodi della query ai nodi del target.

Per creare la sequenza ordinata di nodi $U = (u_1, u_1, \dots, u_n)$ si devono scegliere prima i **nodi più vincolanti**, per rendere la ricerca più veloce.

Al passo i , il nodo che si sceglie ha un **grado molto alto** e un **elevato numero di connessioni** con nodi già presenti nell'ordinamento U^{i-1} .

Il punteggio in un nodo candidato c da inserire è definito sulla base di 3 insiemi:

- $V_{adiac}(i)$: insieme dei nodi U^{i-1} e adiacenti a v ;
- $V_{conn}(i)$: insieme dei nodi U^{i-1} adiacenti ad almeno un nodo non appartenenti a U^{i-1} , che è connesso a v ;
- $V_{rem}(i)$: insieme dei nodi non presenti in U^{i-1} e non connessi a U^{i-1} che sono connessi a v .

Il prossimo elemento di U sarà quello che:

- 1) **massimo valore di $|V_{\text{adiac}}(i)|$;**
- 2) **massimo valore di $|V_{\text{conn}}(i)|$;**
- 3) **massimo valore di $|V_{\text{rem}}(i)|$.**

Si scende al livello inferiore in caso di parità, se ci sono parità anche al livello 3 allora il nodo è scelto arbitrariamente.

Un'ottimizzazione dell'algoritmo RI è l'algoritmo RIDS che consiste nel pre-calcolare, per ogni nodo query q il relativo **dominio di compatibilità**, ossia l'insieme dei punti che potrebbero essere mappati con q . Quindi la regola di Ullman, ossia che il grado di query è inferiore o uguale al grado del nodo del grafo di target, è applicata una sola volta.

Subgraph matching in database di grafi

Dato un database di grafi e un grafo query, si devono trovare tutti i grafi che contengono il grafo query come sotto-grafo.

La soluzione banale prevede nell'applicare uno degli algoritmi di subgraph matching a tutti i grafi del database. È l'operazione più intuitiva da compiere, ma la meno efficiente dato che il supgraph matching è un problema NP-completo.

Per una maggiore efficienza si dovrebbero indicizzare i grafi del database e la query. L'indicizzazione è una rappresentazione più semplice dei grafi e viene svolta off-line una volta sola. L'indicizzazione può essere di due tipi:

- **indicizzazione basata su feature:** ogni grafo viene rappresentato tramite un insieme di feature -> cammini, alberi(cammini più lunghi aciclici), sottografi frequenti -> questi indici creano indici simili a "vocabolari" di caratteristiche dei grafi -> i grafi che non contengono tutte le feature vengono filtrati prima del matching
- **indicizzazione non basata su feature:** ogni grafo mappato in uno spazio metrico(si usa la distanza come misura di filtraggio) e inserito in alberi in cui fare la ricerca in cui fare la ricerca -> adatto a database in cui vengono fatti update frequenti.

Nel caso delle estrazioni delle feature, i cammini e gli alberi sono più facili da estrarre.

Quando ci basiamo sulle feature i passi da seguire sono:

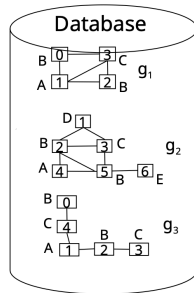
- **pre-processing:** si estraggono tutte le features che contiene ogni grafo;
- **filtering:** dalla query si estraggono le features e si crea un sottoinsieme di grafi candidati che contiene tutte le features della query al proprio interno;
- **matching:** si applica un algoritmo di graph matching per ogni grafo, verificando se contiene la query.

Per velocizzare il filtering si può utilizzare l'indicizzazione inversa, dove ad ogni feature si associa una lista di grafi che la contengono.

L'algoritmo SING(Subgraph search In Non-homogeneous Graphs) è un algoritmo di supgraph matching basato sui cammini di nodi. Oltre alla frequenza di ogni feature associa il nodo da cui parte.

- 1) **Filtering 1** -> scarta subito i grafi che non hanno abbastanza feature;
- 2) **Filtering 2** -> guarda da dove iniziano i cammini di tutte le feature, se non partono dal nodo previsto si scarta il grafo;
- 3) **Matching** -> si fa il matching vero e proprio per vedere se il grafo G contiene il sotto-grafo query.

Frequent subgraph mining



L'obiettivo del **frequent subgraph mining** è trovare **sotto-grafi frequenti** in un dataset di grafi.

Si devono identificare tutti i sotto-grafi che occorrono frequentemente e si conta il numero di grafi che li contengono, ossia calcolare il **supporto**.

Come nel caso degli item-set frequenti, si pone una soglia e se il supporto è **maggiore o uguale a minSup(soglia)** allora il **sottografo è frequente**.

Solitamente però non si fa il conteggio assoluto(supporto), ma si fa riferimento al conteggio relativo, ossia la frequenza di un sotto-grafo in tutti i grafi. La formula per calcolare la frequenza relativa è la seguente:

$$\frac{\text{supporto sottografo}}{\text{numero grafi presenti}}$$

In questo caso la soglia minima σ è una percentuale. Il sotto-grafo è frequente se è presente almeno nel $\sigma\%$ dei grafi del database.

Rispetto alla ricerca di item-set frequenti, la ricerca di subgraph frequenti pone sfide più complesse. In particolare, è necessario affrontare due problemi importanti:

- la strategia da adottare per ridurre la complessità e migliorare l'efficienza dell'algoritmo;
- evitare le ridondanze nella generazione dei candidati, dato che lo stesso sotto-grafo può essere prodotto in modi diversi a causa dell'isomorfismo dei grafi;

Esistono due tipi di algoritmi per il frequent subgraph mining, ossia:

- 1) **Algoritmi Apriori**: basati sulla regola **Apriori**(se è sotto-grafo è frequente allora tutti i sotto-sotto-grafi sono frequenti) che permette di non generare candidati inutili. La **generazione dei candidati** avviene tramite **join di sotto-grafi frequenti** che inizialmente sono piccoli sotto-grafi, composti da un arco. Se sono frequenti verranno uniti ad altri sotto-grafi che condividono lo stesso prefisso tranne l'ultimo arco.
- 2) **Algoritmi pattern-growth**: a differenza degli algoritmi Apriori, non si fanno join, ma aggiungono nuovi candidati(estendono) al sotto-grafo frequente per costruirne uno più grande e si verifica se è frequente nel database.

In tutti e due tipi di algoritmi si segue il **principio bottom-up**, ossia che si parte da sotto-grafi più piccoli arrivando a sotto-grafi più grandi.

L'approccio Apriori è utilizzato perché più semplice, chiaro e funziona bene quando il database non è enorme o i grafi sono piccoli, ma è molto inefficiente. In database più grandi o con grafi estesi si utilizzano gli algoritmi pattern-growth che sono più complessi ma scalano meglio il problema.

Algoritmo FSG(Apriori-Like)

La regola Apriori per filtrare i candidati è la seguente:

il supporto di un sotto-grafo S in un database di grafi non può essere maggiore del supporto dei sotto-grafi di S.

Quindi se S è un sotto-grafo frequente allora tutti i sotto-grafi di S sono frequenti, viceversa se un sotto-grafo G non è frequente, allora tutti i sotto-grafi contenenti G non sono frequenti.

La frequenza di un grafo è "monotona", ossia: un grafo più grande è frequente solo se tutti i suoi sotto-grafi lo sono.

L'algoritmo FSG è il seguente:

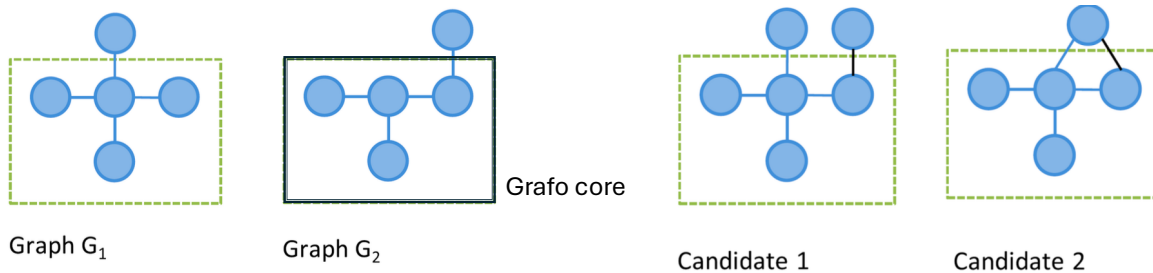
- 1) Calcola i nodi e archi frequenti nel database di grafi e aggiungili all'insieme finale dei risultati O.
- 2) Per $k \geq 3$ (for):
 - a. **(Candidate Generation)** A partire dall'insieme dei sottografi frequenti con $k-1$ archi (la prima iterazione ci saranno 0 archi), costruisci dei sotto-grafi candidati con k archi (alla prima iterazione si costruiranno sotto-grafi composti da un arco e due nodi).
 - b. **(Pruning)** Applica la regola Apriori, rimuovendo i candidati che contengono almeno un sotto-grafo contenente $k-1$ archi non frequente.
 - c. **(Counting)** Calcola il supporto di ogni candidato C e aggiungi all'insieme finale O.
- 3) Restituisci l'insieme finale O.

Molto facile da capire:

- costruisci tutti i possibili candidati con k archi;
- rimuovi quelli che contengono sotto-grafi non frequenti;
- calcola il supporto dei grafi rimanenti e mettili nell'insieme finale.

Infine, si restituisce l'insieme finale, quando si arriva alla k -esima iterazione.

La costruzione dei possibili candidati si fa solo se i due sotto-grafi hanno almeno un sotto-grafo con archi in comune, chiamato **grafo core**.



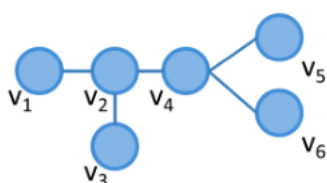
Come possiamo osservare dall'immagine la join tra due sotto-grafi può generare grafi diversi. Gli scenari possibili sono:

- i due sotto-grafi differiscono per un nodo avente la stessa etichetta $\rightarrow$ si hanno join multipli equivalenti;
- i grafi hanno più automorfismi $\rightarrow$ si ha ambiguità durante l'inserimento del nuovo arco (dato che ad esempio A-B-C sono intercambiabili);
- i due sotto-grafi hanno più grafi core in comune $\rightarrow$ si crea una join per ogni sotto-grafo;

Avendo F^k l'insieme dei sotto-grafi frequenti, per ogni core condiviso da G_i^k e G_j^k si fa la join (anche in caso di automorfismi e più core in comune) e si aggiungono i sotto-grafi ottenuti nell'insieme C^{k+1} dei sotto-grafi candidati.

Dato che durante la generazione dei candidati si possono generare sotto-grafi ridondanti (isomorfi), si calcola un codice univoco ossia la **forma canonica**. Se si hanno forme canoniche uguali allora si rimuovono le ridondanze.

La forma canonica generata è la **matrice di adiacenza $A(G)$** come nell'esempio:



$$A(G) = \begin{matrix} & \begin{matrix} v_1 & v_2 & v_3 & v_4 & v_5 & v_6 \end{matrix} \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \end{matrix}$$

$$A(G) = \begin{matrix} & v_1 & v_2 & v_3 & v_4 & v_5 & v_6 \\ \begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{matrix} & \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix} \end{matrix}$$

Dalla matrice di adiacenza si può ottenere la stringa di adiacenza, concatenando le righe della triangolare superiore (per grafi indiretti).
 $S(G) = 100001100000110$
 La parte inferiore non si prende perché speculare alla superiore.

In un grafo con n nodi abbiamo $n!$ **rinumerazioni** (permutazioni), che producono stringhe di adiacenza diverse. Generare tutte le permutazioni avrebbe un costo oneroso, per questo motivo si sfruttano i gradi dei nodi oppure le etichette per generare le stringhe di adiacenza. Esempio nodi con grado 3 poi 2 e poi 1.

Il calcolo della forma canonica può essere fatto seguendo una serie di step, quali:

- **Raggruppamento** dei nodi in gruppi **sulla base del grado** (es. $\{A,B\} \rightarrow$ grado 3, $\{C\} \rightarrow$ grado 2, $\{D,E,F\}$ grado 1);
- Si **partizionano** i nodi dei gruppi **in etichette** (es. $\{A\}, \{B\}$) così i nodi non intercambiabili vengono separati;
- Si permutano solo i nodi delle partizioni ottenute attraverso le etichette;
- Si sceglie infine la stringa lessicograficamente più piccola;

Per ogni sotto-grafo ottenuto si effettua il controllo delle frequenze attraverso l'algoritmo Apriori, se non è frequente si esclude.

TID(transaction identifier) list: ogni sotto-grafo frequente S memorizza la lista dei grafi che lo contengono.

Intersezione: per un candidato $(k+1)$ sotto-grafo, calcola l'intersezione delle TID list dei suoi k sotto-grafi.

Filtro: se la lista intersezione ha meno elementi del supporto minimo $\rightarrow$ scarta il candidato; altrimenti fai il subgraph matching solo sui grafi della lista intersezione.

Algoritmo GSPAN(Pattern-Growth like)

L'**approccio Apriori** è basato sulla visita dello spazio delle soluzioni in ampiezza. Quindi, prima di generare i candidati con $k+1$ archi dovranno essere calcolati tutti i sotto-grafi frequenti con k archi.

I **pattern-growth** si basano sulla visita dello spazio delle **soluzioni in profondità**, quindi se un sotto-grafo con k archi è frequente si estende (si passa a $k+1$ anche se tutti i grafi con k archi non sono stati generati).

L'allocazione richiesta per gli algoritmi pattern-growth è minore rispetto per gli Apriori, però la fase di pruning è più complicata.

L'algoritmo proposto per l'approccio pattern-growth è la ricerca mediante **gSpan**. Quest'algoritmo permette di associare ad ogni sotto-grafo un codice, chiamato **codice DFS** (calcolato attraverso la visita DFS (depth-first)), e funge da **forma canonica**.

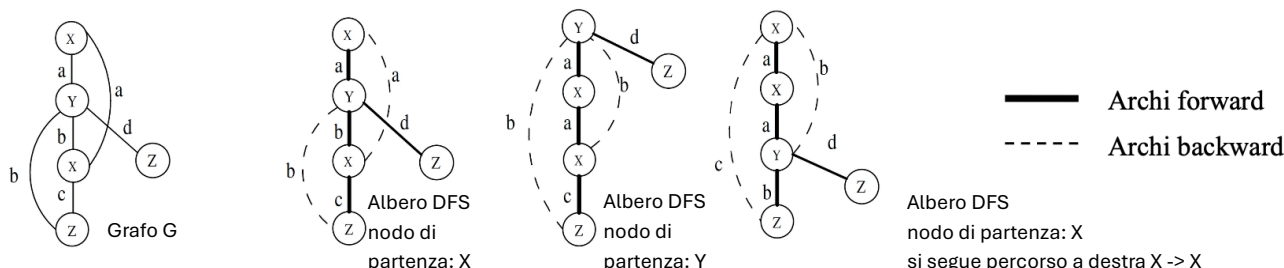
Tutti i possibili sotto-grafi vengono organizzati in un albero, detto DFS code tree, dove:

- Ogni nodo dell'albero è un DFS code minimo (forma canonica associata ad ogni sotto-grafo);
- I figli sono estensioni del nodo padre (aggiunta di archi o nodi);
- Ordinamento definito secondo l'ordine lessicografico dei codici DFS.

La visita **DFS va in profondità** (partendo da un nodo) finché può e quando un nodo non ha più archi da esplorare, si torna al nodo precedente v' . Da v' si continua ad esplorare gli archi rimasti. DFS termina quando tutti i nodi e gli archi sono stati visitati.

La visita DFS di un grafo G produce un **albero T**(albero DFS), dove i nodi sono quelli di G, gli archi sono gli archi di G esplorati durante la visita.

Gli archi di G contenuti in T sono chiamati **archi forward**, invece, i restanti sono **archi backward**.



Nell'esempio sopracitato possiamo vedere il grafo G(primo a sinistra) con dei cicli. I successivi sono alberi DFT (T) che mostrano come cambia la struttura fissando diversi nodi di partenza o ordini di esplorazione diversa.

Seguendo un ordine di esplorazione di visita dei nodi della DFS, è possibile costruire un codice DFS, formato da una sequenza ordinata di archi backward e forward stabilita secondo criteri precisi. Ogni arco dell'esplorazione è rappresentato nel codice DFS attraverso una quintupla:

$$(t(u), t(v), \text{lab}(u), \text{lab}(u,v), \text{lab}(v))$$

Dove:

- t indica il **tempo di visita** di un nodo(posizione di v nell'ordine di visita);
- lab è l'**etichetta** di un nodo($\text{lab}(u)$) o di un arco($\text{lab}(u,v)$) -> se non ci sono etichette, si omettono nel codice.

Nell'ordine del codice della DFS:

- si visitano gli **archi backward**;
- si visitano gli **archi forward**;

Nell'indecisione per archi simili, si dà precedenza archi più anziani, quelli con il tempo di visita minore.

edge no.	(b) α	(c) β	(d) γ
0	(0, 1, X, a, Y)	(0, 1, Y, a, X)	(0, 1, X, a, X)
1	(1, 2, Y, b, X)	(1, 2, X, a, X)	(1, 2, X, a, Y)
2	(2, 0, X, a, X)	(2, 0, X, b, Y)	(2, 0, Y, b, X)
3	(2, 3, X, c, Z)	(2, 3, X, c, Z)	(2, 3, Y, b, Z)
4	(3, 1, Z, b, Y)	(3, 0, Z, b, Y)	(3, 0, Z, c, X)
5	(1, 4, Y, d, Z)	(0, 4, Y, d, Z)	(2, 4, Y, d, Z)

Il **codice minimo** sarà quello che sarà **lessicograficamente più piccolo**. Possiamo vedere che si sceglie (d) perché nella riga 0 si ha:

(0, 1, X, a, Y)	(0, 1, Y, a, X)	(0, 1, X, a, X)
-----------------	-----------------	-----------------

Perché a parità di 0,1 si confrontano le etichette e X,X è la minore.

Come accennato prima il **DFS code tree** è un albero, rappresentante lo spazio dei candidati esaminati da gSpan. I nodi rappresentano un grafo identificato dal codice DFS minimo.

Per generare un candidato in gSpan con $k+1$ archi, partendo da un sotto-grafo frequente con k archi, si aggiunge un solo arco. Per evitare eventuali ridondanze si aggiunge il nuovo arco secondo dei criteri, ossia:

- **partire da un nodo** che sta nel cammino **più a destra**(convenzione cammino **right-most**) nell'albero DFS associato al codice DFS minimo(così si ha sempre un punto di partenza e un cammino da seguire prefissato(contestualizzato all'albero)).
- **partendo dal nodo foglia del cammino più a destra si aggiungono gli archi** del cammino(evitando ridondanze)

Quando si generano ridondanze ossia un grafo dell'albero è uguale ad un altro si fa pruning su uno dei due.

Reti Neurali

Una **rete neurale artificiale** simula il comportamento dei neuroni del cervello umano, dove ogni neurone riceve in input segnali da diversi altri neuroni (li elabora e si attiva) e produce segnali in output. Il segnale di output prodotto può essere:

- risultato finale della rete;
- segnale d'input per altri neuroni.

Ogni nodo riceve n valori d'input pesati, li somma e applica una **funzione di attivazione**. La funzione di attivazione decide se il neurone si deve attivare (ad esempio una funzione sigmoide) e quale sarà l'output trasmesso.

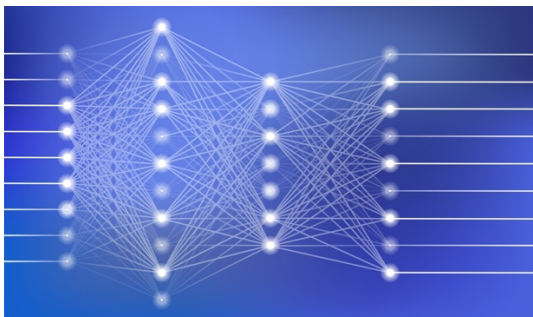
Una **rete neurale** è composta da **layer o strati**, ossia un insieme di neuroni che svolgono una specifica funzione nella fase di elaborazione dei dati. I tre layer sono:

- **input layer** -> è il primo layer della rete e ha un numero di nodi pari alla dimensione dell'input. *Non elabora informazioni, ma si limita a riceverle e passarle allo strato successivo;*
- **hidden layer** -> è lo stato interno alla rete i cui nodi:
 - riceve valori dai nodi dell'input layer;
 - applica i pesi agli input;
 - somma i contributi e aggiungono bias;
 - applica una funzione di attivazione;
 - trasmette gli output al layer successivo.

Una rete che ha molti hidden layer è chiamata **Deep Neural Network (DNN)**. L'aggettivo deep indica la profondità strutturale, ossia la presenza di molti livelli intermedi.

- **output layer** -> è l'ultimo layer della rete, e può essere formato da uno o più nodi che **restituiscono in output un valore** a seconda del problema:
 - 1 nodo -> regressione;
 - più nodi -> classificazione multi-classe;

Le reti neurali operano su strutture dati chiamate **tensori**, ossia **array a n dimensioni**, es. immagini -> array 3d perché consideriamo altezza x larghezza x colori.



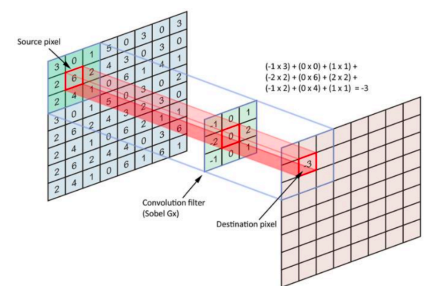
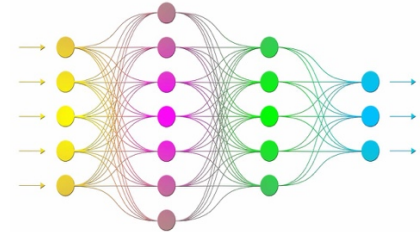
Una **rete neurale** è detta **densa**, se ogni nodo di un layer riceve tutti gli output dei nodi del layer precedente (**layer totalmente connessi**).

Oltre al modello denso, esistono altre modalità di connessione tra i layer:

- **random** -> fissato un numero m , ogni nodo del **layer corrente riceve input da m nodi scelti in modo casuale dal layer precedente** (rete più facile da allenare dato che il numero di connessioni sono ridotte);
- **pooled** -> i **nodi di un layer vengono partizionati in k cluster distinti**. Il layer successivo sarà composto da k nodi, uno per ogni cluster creato. Questo permette di ridurre la dimensionalità e di raggruppare informazioni simili **mantenendo le informazioni aggregate**.

Un esempio di rete neurale diversa da quelle sopraelencate è la **connessione convoluzionale**, ossia una **rete con connessioni locali e pesi condivisi**.

Ogni neurone del layer è collegato solo ad una regione (l'intorno del punto) e condivide lo stesso peso con il resto dei neuroni.



Quando si progetta una rete neurale si deve stabilire:

- **numero di hidden layer** -> è uno dei problemi paralleli al trade-off bias/varianza, infatti:
 - con **pochi layer** il modello rischia di essere **molto semplice** e di fare **underfitting**;
 - con **tanti layer** ha una **capacità maggiore** ma si può incorrere in **overfitting**.Solitamente si parte semplice e si aumenta la complessità solo se serve.
- **numero di nodi per layer** -> dipende dal tipo di problema e dalla quantità di dati -> le strutture comuni sono piramidali (si parte da molti input e si ritornano pochi output).
- **tipo di connessioni tra layer** -> dipende dall'obiettivo della rete e dal tipo di dati.
- **tipo di funzione di attivazione per ogni layer** -> dipende dal tipo di layer, da delle considerazioni numeriche sui dati e dal tipo di problema da risolvere.

Definita la struttura, il modello deve essere addestrato su un **training set per calcolare i valori ottimali dei pesi**.

Si definisce una **funzione di costo** (function loss F), il cui valore restituito dalla funzione viene minimizzato dai pesi.

La funzione di attivazione di un nodo n è la funzione F che determina la risposta $F(z)$ prodotta dal nodo n a partire da $z = \vec{w} \cdot \vec{x} + b$, dove:

- $\vec{w}$ è il **vettore dei pesi** associati;
- $\vec{x}$ è il **vettore degli input**;
- b è la costante di **bias**;

Le funzioni di attivazione sono uguali per uno stesso layer.

Una funzione di attivazione deve essere:

- **continua** (nessun salto nei valori) e **differenziabile** (derivabile) in ogni punto;
- la **derivata** della funzione **non deve saturare** (tende a 0) nel suo dominio, poiché derivate troppo piccole nella ricerca dei pesi ottimali portano ad uno **stallo del processo**;
- la **derivata non deve esplodere** (tendere ad infinito), poiché causa instabilità numerica nella ricerca dei pesi ottimali.

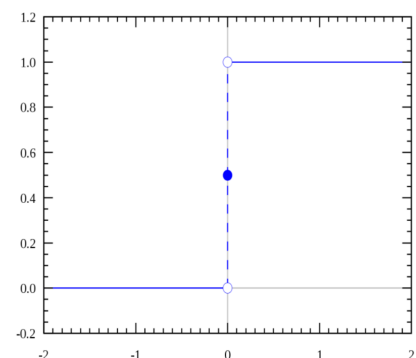
Un esempio di funzione che non rispetta il 2° e il 3° punto, ossia che satura o esplode, è la **funzione step**. Posta una soglia, la funzione step permette di determinare il valore binario in uscita.

$$\text{step}(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{altrimenti} \end{cases}$$

Questa funzione può essere utilizzata per costruire un classificatore discriminativo binario o multi-classe.

Il perceptron è un esempio di classificatore binario che restituisce 1 se l'elemento appartiene ad una determinata classe 0 altrimenti.

Nell'output layer, nel caso di classificazione multi-classe, occorre creare un perceptron per ogni classe.



La **funzione logistica** appartiene alla classe delle funzioni sigmoidee, aventi una curva a forma di S.

Abbiamo che:

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1 + e^x}$$

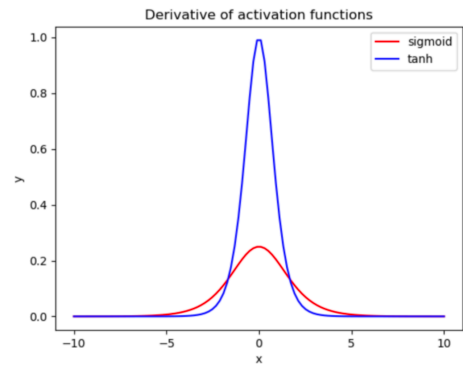
Nel contesto della classificazione può essere utilizzata nel layer di output per restituire la probabilità di appartenenza di una tupla a ciascuna classe. La funzione logistica ha dei problemi dato che la derivata satura.

Una versione shiftata e scalata della funzione logistica è la **tangente iperbolica** e ha valori compresi tra -1 e 1. La formula della tangente iperbolica è:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Possiamo vedere come confrontando le derivate delle due sigmoidi, si ha che la derivata si satura nella logistica ma non nella tangente iperbolica.

$$\begin{aligned}\sigma'(x) &= \sigma(x)(1 - \sigma(x)) \\ \tanh'(x) &= 1 - \tanh^2(x)\end{aligned}$$



La **funzione softmax**, a differenza delle funzioni sigmoidi, non opera su un singolo valore di un vettore ma su tutto il vettore.

Quindi se si ha un vettore $\vec{x} = (x_1, x_2, \dots, x_n)$, allora la funzione di softmax è definita come $\vec{\mu} = (\mu(x_1), \mu(x_2), \dots, \mu(x_n))$, dove:

$$\mu(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

La funzione softmax restituisce una probabilità tra 0 e 1, e la somma è uguale a 1. Il regressore softmax potrebbe avere problemi con esponenti molto grandi, ma sottraendo una costante c, si evita l'esplosione di valori.

$$\mu(x_i) = \frac{e^{x_i - c}}{\sum_{j=1}^n e^{x_j - c}}$$

Un'ulteriore funzione di attivazione è la **rectified linear unit (ReLU)**, ossia una funzione che prende spunto dai raddrizzatori a singola semionda, che trasformano un segnale alternato in un segnale unidirezionale (sempre positivo o sempre negativo).

$$f(x) = \max(0, x) = \begin{cases} x & \text{se } x > 0 \\ 0 & \text{se } x \leq 0 \end{cases}$$

La **ReLU** viene utilizzata perché converge più velocemente, raggiungendo la performance ottimale con minore costo computazionale rispetto alle funzioni che utilizzano sigmoide.

La derivata è $f'(x) = \begin{cases} 1 & \text{se } x > 0 \\ 0 & \text{se } x \leq 0 \end{cases}$ che converge molto più velocemente.

Per evitare di porre uguale a 0 (saturazione) valori sotto lo zero si utilizza la **ELU** o **exponential linear unit**, dove:

$$f(x) = \max(0, x) = \begin{cases} x & \text{se } x > 0 \\ \alpha(e^x - 1) & \text{se } x \leq 0 \end{cases}$$

Questo permette di far apprendere ai neuroni anche in situazioni dove $x < 0$. Il ramo negativo ottenuto da questa funzione è modificabile attraverso l'iper-parametro α .

Le funzioni loss citate prima sono le funzioni di costo, usate nella fase di apprendimento per quantificare la differenza tra il valore predetto e valore presente nel training set. I pesi ottimali sono quelli che minimizzano la loss function.

La scelta della loss function dipende dal tipo di problema che si deve affrontare. Infatti, abbiamo che:

- **regression loss** -> per un valore reale in output o un vettore di valori reali (MSE/MAE);
- **classification loss** -> output distribuzione di probabilità, la probabilità è legata alla classe di appartenenza (cross-entropy)

Per la regression loss utilizziamo funzioni quali:

- **squared error loss:** funzione che calcola la differenza tra la predizione e la realtà e poi la eleva al quadrato. La funzione è la seguente: $L(y, \hat{y}) = (y - \hat{y})^2$. Questa regression loss penalizza gli errori molto grandi, elevandoli al quadrato, utile al modello ma lo rende sensibile agli outliers.
- **huber loss:** funzione ibrida che serve per risolvere il problema della sensibilità degli outliers. Per errori elevati la funzione diventa lineare. La funzione è la seguente:

$$L_{\delta} = \begin{cases} (y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \sigma \\ 2\sigma(|y - \hat{y}| - \frac{1}{2}\sigma) & \text{altrimenti} \end{cases} \text{ dove } \sigma \text{ è l'iper-parametro.}$$

- **mean square error:** la funzione che calcola l'errore quadratico medio e semplifica la derivata (che serve nella backpropagation quando applichiamo la discesa del gradiente). La funzione è la seguente:

$$MSE(P, T) = \frac{1}{N} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **root mean square error:** la funzione applica all'MSE la radice quadrata, rendendo più leggibile quanto il modello si sta sbagliando. La funzione è la seguente:

$$RMSE(P, T) = \sqrt{MSE}$$

Per la classification loss si considera che si hanno $C_1, \dots, C_n$. Dato un vettore di input $\vec{x}$ e il relativo vettore di probabilità di appartenenza alla classe C_i ossia $\vec{p}$. Supponendo che la rete neurale produca una distribuzione di probabilità $\vec{q}$, la classification loss indica la distanza tra $\vec{p}$ e $\vec{q}$.

Le funzioni utilizzate per la classification loss sono:

divergenza kullback-Leibler (cross entropy): la funzione considera il teorema di Shannon che dice che il numero medio di bit per rappresentare un messaggio è l'entropia $H(\vec{p})$. Riadattandola con i valori predetti, abbiamo che:

$$C(\vec{p} || \vec{q}) = - \sum_{i=1}^n p_i \log q_i$$

Partendo da questo presupposto possiamo calcolare la funzione di costo come segue:

$$K(\vec{p} || \vec{q}) = C(\vec{p} || \vec{q}) - H(\vec{p}) = \sum_{i=1}^n p_i \log \frac{p_i}{q_i}$$

Indipendentemente dal modello scelto, il training consiste nel trovare i pesi che minimizzano l'average loss su un training set, evitando overfitting (si adatta anche ai rumori e non riesce a generalizzare su dati di training set).

Il **gradiente** durante la minimizzazione dell'average loss è importante, dato che ci indica la direzione massima di pendenza e ci permette di aumentare o diminuire l'errore. Infatti, il gradiente contiene tutte le derivate prime parziali di y rispetto a tutti i valori di x .

$$\nabla_{x \rightarrow y} = \left(\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_n} \right)$$

Quando y non è uno scalare, ma è un vettore, si utilizza la matrice Jacobiana che mette in relazione tutte le componenti $\vec{y}$ rispetto ciascuna componente $\vec{x}$.

$$J_{\vec{x}}(\vec{y}) = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_1} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_1}{\partial x_n} & \dots & \frac{\partial y_m}{\partial x_n} \end{pmatrix}$$

Il metodo di **discesa del gradiente**, come accennato prima, è l'operazione che permette di cercare la direzione massima di discesa del valore della funzione e aggiorna i valori seguendo la direzione. Tutto ciò è fatto in modo iterativo fino a quando i valori del vettore non cambiano significativamente.

Il **learning rate η** indica la **velocità di convergenza al valore ottimale**.

- Se il learning rate è basso, ci vogliono molte interazioni per convergere al valore ottimale però il modello impara;
- Se il learning rate è molto alto in pochissime interazioni si cercherà di convergere, ma si può incorrere in oscillazioni dei valori, ergo divergenza involontaria.

La soluzione ottimale è quella di avere un learning rate alto all'inizio e quando si è vicini alla soluzione si deve applicare un fattore β , compreso tra 0 e 1, al learning rate in modo tale da diminuire la velocità di discesa.

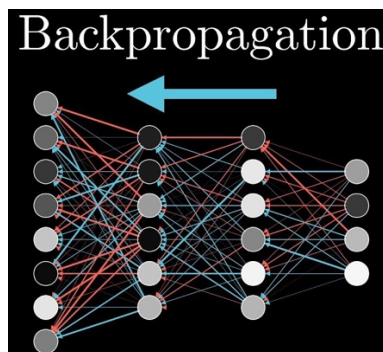
I pesi dovranno essere inizializzati in maniera random e l'intervallo scelto è o $[0,1]$ o quello di una distribuzione gaussiana. La scelta dei valori del peso in modo random deve prevedere che essi siano tutti equiprobabili cosicché non si ha lo stesso aggiornamento del gradiente.

Il **Stochastic gradient descent** è una variante diversa della discesa del gradiente tradizionale, che prevede il calcolo di gradienti sequenziali (uno dopo l'altro), su campioni del dataset. Questo permette di calcolare il gradiente molto più velocemente in dataset molto grandi.

Il processo che permette di calcolare i gradienti in una rete neurale è la **backpropagation**.

Dopo aver calcolato la loss function, la rete viene vista come un **grafo computazionale aciclico (DAG)** in cui **ogni peso(nodo) contribuisce al risultato finale**. La backpropagation applica la **regola della catena** (chain rule) per calcolare, per ciascun peso, il gradiente della loss rispetto a quel peso. In questo modo è possibile sapere come ogni parametro ha influenzato l'errore.

Durante la fase di ottimizzazione (ad esempio con SGD, Adam, ecc.), questi gradienti vengono utilizzati per aggiornare i pesi nella direzione che riduce la loss, migliorando progressivamente le prestazioni della rete.

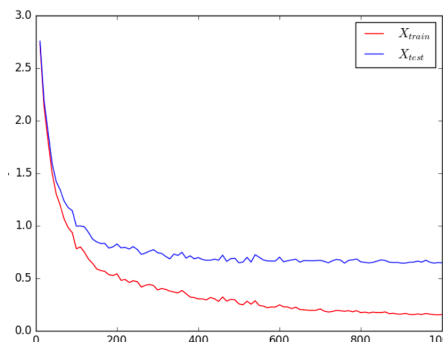


Tecniche di regolarizzazione

Le **tecniche di regolarizzazione** sono delle tecniche che permettono di **evitare l'overfitting**, cioè discostano la rete dall'abituarsi ai valori di training (anche agli outliers) dato che poi non riesce a generalizzare sui valori di test set.

È preferibile quindi avere un bias alto sui valori del training set piuttosto che sui valori del test set. Inoltre, si cerca di diminuire il valore della loss function sui valori sia del training set e sia test set ad ogni epoca.

Soluzioni con pesi che hanno il valore assoluto piccolo, sono preferibili a soluzione che hanno pesi con valore assoluto molto alto e per far ciò si può anche forzare il gradiente a preferire pesi piccoli.



Il gradiente viene forzato a favorire soluzioni con pesi in valore assoluto piccolo attraverso una penalità nella loss function L_0 usata nel modello.

Un esempio di penalità aggiuntiva alla loss function sono le due regolarizzazioni:

- **Regolarizzazione L1(regressione Lasso):** aggiunge come termine di penalità la somma dei valori assoluti dei pesi. La formula è la seguente:

$$L = L_0 + \alpha \sum_{i=1}^k |w_k|$$

- **Regolarizzazione L2(regressione Ridge):** aggiunge come termine di penalità la somma dei quadrati dei pesi. La formula è la seguente:

$$L = L_0 + \alpha \sum_{i=1}^k w_k^2$$

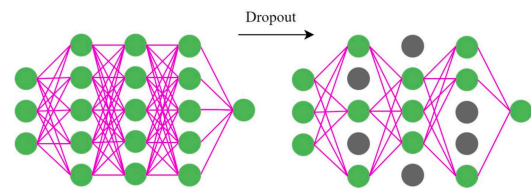
Si ricorda che α è un iper-parametro detto **parametro di regolarizzazione**, compreso tra 0 e 1, che controlla l'intensità della penalità: più è grande, più il modello tende a ridurre i pesi.

La differenza tra i due tipi di regressione è che:

- L1 tende a portare alcuni pesi a 0, realizzando una vera e propria selezione delle feature;
- L2 non annulla i pesi, ma favorisce soluzioni più stabili e meno sensibili al rumore, senza annullare completamente le feature considerate.

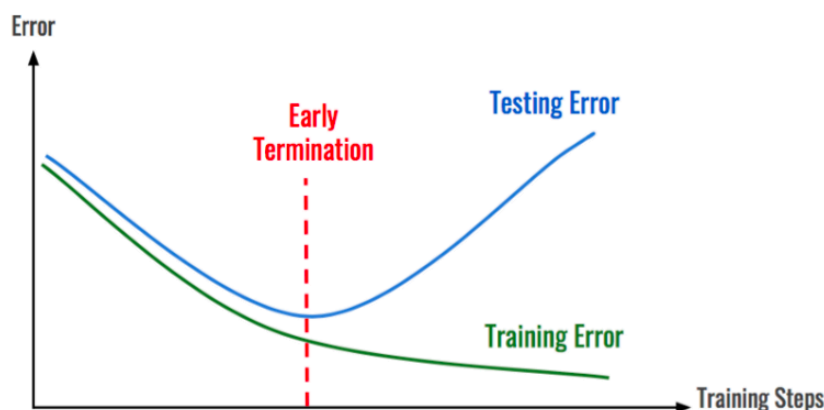
Per ridurre ancor più l'overfitting si utilizza un'ulteriore tecnica di regolarizzazione, ossia il dropout. Questa tecnica ad ogni epoca(batch) disattiva casualmente i neuroni all'interno della rete(dropout rate), favorendo due vantaggi:

- Semplificazione della rete;
- Generalizzazione migliore del modello:
 - non dipende da singoli dati;
 - impara rappresentazioni più robuste;
 - diventa meno sensibile agli outliers.



Quando cerchiamo di diminuire la loss function nel training set, essa diminuirà all'aumentare delle iterazioni e del tempo di addestramento, dato che la monotonia è decrescente.

Nel test set, la loss function potrebbe raggiungere un minimo e poi tornare a crescere. Per evitare che cresca, si potrebbe stoppare prematuramente il training. Per questo motivo, si utilizza un'ulteriore porzione del dataset, ossia il validation set, che oltre a far stoppare prematuramente(**early stopping**) il training serve per la ricerca degli iper-parametri.



Tipi di reti neurali

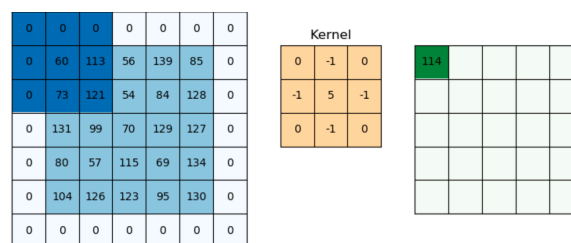
Ci sono diversi tipi di reti neurali e cambiano a seconda degli obiettivi che si vogliono raggiungere e/o ai diversi tipi di implementazione.

Il tipo più semplice e fondamentale di rete artificiale è la **Feed-Forward Neural Network**, dove l'informazione tra i layer scorre in **una sola direzione(avanti)**. In sostanza, non sono ammessi cicli all'interno della rete neurale e solitamente sono utilizzate per problemi di classificazione o regressione.

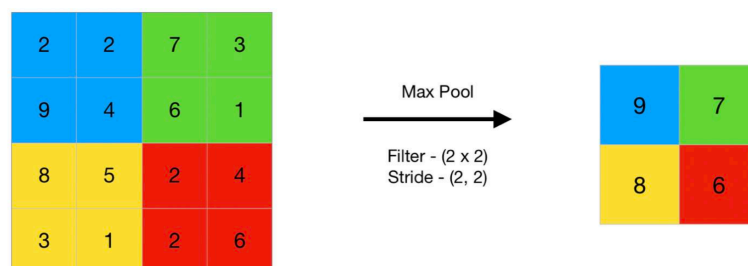
Per estrarre caratteristiche dai dati, in particolare dalle immagini, si utilizza una **Convolutional Neural Network(CNN)**, vale a dire una rete neurale che utilizza **layer convoluzionali**.

I layer convoluzionali (*studiati a interazione-> si prende un kernel 3*3 (dispari in generale) e si applica ad ogni singolo punto dell'immagine*) applicano dei filtri(kernel) per individuare pattern come bordi, texture o forme.

Noto attraverso lo stride (quantità di spostamento del kernel all'interno dell'immagine) che applicando un filtro ad ogni punto dell'immagine produce un output più piccolo, si può utilizzare la tecnica dello **zero padding**. Questa tecnica consiste nell'aggiungere alla matrice di input righe e colonne esterne di 0.



Oltre i layer convoluzionali si utilizzano i **layer pooled** per ridurre la dimensione della feature map, mantenendo le informazioni più rilevanti e rimuovendo solo i parametri superflui. Scorre come i layer convoluzionali in orizzontale e in verticale un filtro, ma seleziona solo o il valore medio o il valore massimo.



Un tipo di rete neurale che è simile alla CNN, ma è progettata per lavorare sui grafi è la Graphs Neural Network. La rete in questione è utilizzata per le reti stradali, social network e sistemi di raccomandazione.

La GNN assegna ad ogni nodo un vettore di rappresentazione che cattura le informazioni:

- caratteristiche del nodo(etichetta e attributi);
- posizione nel grafo;
- informazioni dei nodi vicini.

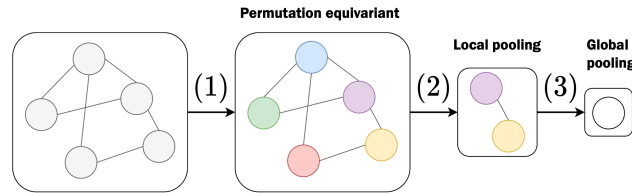
Questa rete neurale serve per scopi come:

- classificazione nodo (prevedere in social network se l'utente è umano o bot);
- classificazione grafo (data la struttura della molecola capire se è tossica o meno);
- predizione di archi (se due nodi potrebbero essere collegati -> due possibili amici nei social network)

Il punto cardine delle GNN è il **message passing**, ossia l'aggiornamento del vettore di rappresentazione attraverso l'aggregazione delle informazioni provenienti dai vicini.

L'architettura generale di una GNN è costituita dai seguenti layer:

- 1) **Permutation equivariant**: aggiorna la rappresentazione dei nodi attraverso il message passing;
- 2) **Local pooling**: si comprime il grafo aggregando alcune informazioni su alcuni nodi;
- 3) **Global pooling**: si comprime l'intero grafo in un unico vettore di rappresentazione.



Un altro tipo di rete è la **Recurrent Neural Network**, vale a dire una rete neurale progettata con dei cicli, dove un **layer può ricorrere più volte**. Questa tipologia di reti neurali è sviluppata per **lavorare con dati sequenziali**, dove l'ordine è importante, per questo motivo erano utilizzate per la generazione di audio e testo.

I layer ricorrenti solitamente fungono da **memoria di stato** e sono **gli strati nascosti**, e si utilizzano per ricordare informazioni passate sulla quale si modellano comportamenti dinamicamente temporale.

I pesi della RNN sono condivisi tra tutti i passi della sequenza e sono rappresentati da 3 matrici principali:

- **U** → connette l'input allo stato nascosto;
- **W** → connette lo stato precedente allo stato corrente
- **V** → connette lo stato nascosto all'output

Lo stato nascosto funziona come una parola che accumula informazioni sulle parole o valori osservati al passo t . La funzione per lo stato nascosto è:

$$s_t = f(Ux_t + Ws_{t-1} + \vec{\beta})$$

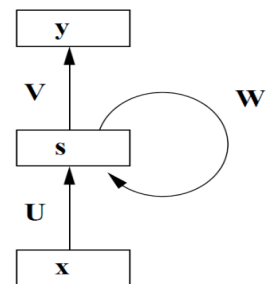
dove:

- **f** è una **funzione di attivazione non lineare**;
- s_0 è inizializzato a zero;
- s_t rappresenta ciò che la **rete neurale ricorda al passo t**.

L'output al passo t viene calcolato dallo stato nascosto: $y_t = g(Vs_t + \vec{c})$

dove:

- **g** è una **funzione di classificazione**;
- $\vec{c}$ è un **vettore di bias**.



Reti neurali di questo tipo hanno una lunghezza fissa per gli stati in input e quindi nel caso i vettori di input abbiano una lunghezza variabile, si può risolvere il problema come segue:

- **zero-padding**: si fissa una lunghezza n massima della sequenza e nel caso in cui le sequenze siano più corte, si riempie con degli 0 fino alla lunghezza prefissata (c'è uno spreco);
- **bucketing**: si raggruppano più sequenze in **bucket**, a seconda delle loro lunghezze, e si costruisce una RNN per ogni lunghezza.

Solitamente si utilizzano le due tecniche insieme, si definiscono pochi bucket e le sequenze più piccole per rientrare all'interno di essi vengono riempite attraverso degli 0.

L'obiettivo delle RNN è dare peso allo storico dell'elaborazione, ma a livello pratico solo gli stati recenti hanno peso maggiore, infatti, informazioni lontane si dissolvono nel tempo.

Durante il training, il gradiente deve attraversare tutti i passi delle sequenze e questo lo rende instabile.

L'instabilità fa sì che l'RNN non riesca ad aggiornare correttamente i pesi legati a dipendenze lunghe, ergo non ricorda informazioni lontane.

Una versione migliore delle RNN sono le reti **Long Short-Term Memory(LSTM)**, che affronta il problema delle **relazioni a lunga distanza**. Questa tipologia di rete si può riassumere come segue:

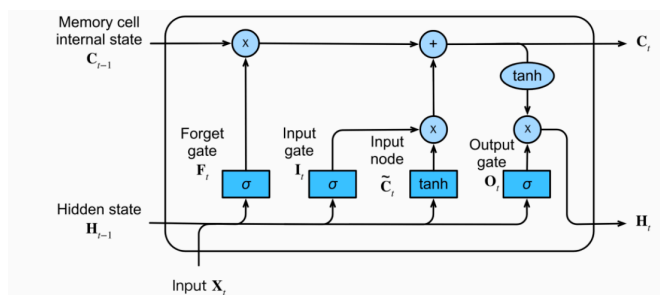
- **Forget**: elimina informazioni non più utili alla memoria;
- **Save**: capacità di salvare informazioni a lungo termine nella memoria long-term (analisi di un testo salvare solo parole chiave);
- **Focus**: capacità di focalizzarsi solo su aspetti della memoria immediatamente rilevanti.

Per realizzare queste proprietà della rete, LSTM introduce una gestione della memoria più controllato, cioè:

- Memoria corrente;
- Memoria a lungo termine;
- **Gate**, ossia porte che decidono cosa ricordare, cosa dimenticare e cosa mostrare come output.

I gate permettono di catturare le relazioni meglio delle RNN.

La struttura di una LSTM è come segue.



σ sono le porte del gate.

Si definisce il **prodotto di Hadamard** come il prodotto punto a punto tra vettori o matrici. Attraverso il prodotto sopracitato si definiscono tutte le funzioni di aggiornamento delle memorie. Sotto sono fornite le formule e la descrizione di esse.

Input gate: calcola quanto della nuova informazione (dall'input corrente) deve essere inserita nella memoria a lungo termine (cell state) della LSTM.	$\vec{I}_t = \sigma(W_I \vec{H}_{t-1} + U_I \vec{X}_t + \vec{b}_I)$
Forget gate: stabilisce quali parti della memoria a lungo termine devono essere mantenute e quali dimenticate.	$\vec{F}_t = \sigma(W_F \vec{H}_{t-1} + U_F \vec{X}_t + \vec{b}_F)$
Candidate Memory: a generare una nuova possibile informazione da inserire nella memoria a lungo termine.	$\vec{\tilde{C}}_t = \tanh(W_C \vec{H}_{t-1} + U_C \vec{X}_t + \vec{b}_H)$
Aggiornamento memoria a lungo termine: definisce come viene aggiornata la cell state (memoria a lungo termine)	$\vec{C}_t = \vec{C}_{t-1} \circ \vec{F}_t + \vec{\tilde{C}}_t \circ \vec{I}_t$
Output gate: decide quale parte della cella di memoria deve essere resa visibile come stato nascosto in quel passo.	$\vec{O}_t = \sigma(W_O \vec{H}_{t-1} + U_O \vec{X}_t + \vec{b}_O)$
Hidden state finale: è lo stato nascosto al tempo $t \rightarrow$ rappresenta ciò che la rete "outputta" in quel passo.	$\vec{H}_t = \vec{O}_t \circ \tanh(\vec{C}_t)$

Una rete neurale utilizzata per la compressione dei dati è la **autoencoder**, che attraverso una quantità minima di informazioni permette di ricostruzione integrale e precisa.

Utilizza due funzioni, quali:

- **encoder** -> trasforma l'input originale in una rappresentazione più piccola, chiamata spazio latente Z. Lo spazio latente contiene solo le informazioni principali dei dati. Ogni vettore all'interno dello spazio latente rappresenta una piccola porzione di dati compressi.
- **decoder** -> ricostruisce l'informazione originale dall'informazione compressa Z.

La funzione di loss si basa sulla differenza tra l'input e l'output ricostruito. La dimensione dello spazio Z non deve essere né troppo piccolo e né troppo grande:

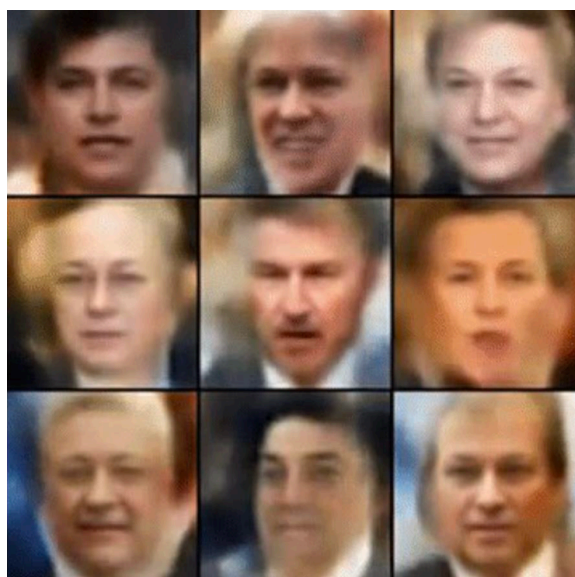
- se la dimensione di Z è piccola, allora si potrebbero ricostruire dati diversi dall'originale;
- se la dimensione di Z è grande, allora la compressione non risulta efficace.

Abbiamo tre principali tipi di autoencoder:

- **autoencoder tradizionale**: comprimere dati mantenendo le informazioni principali;
- **autoencoder denoising**: riesce a ricostruire i dati scartando il rumore -> in fase di training si aggiunge rumore ai dati con lo scopo di addestrare un modello;
- **autoencoder sparsi**: permette di generare codici sparsi (con molti 0), per alleggerire l'informazione, riducendo l'overfitting e rendendolo quindi meno sensibili al rumore.

Una variante dell'autoencoder è la **variational autoencoder**, che non si limita a comprimere e ricostruire i dati, bensì modella la distribuzione dei dati nello spazio latente. L'encoder trasformerà i dati in input in parametri di una distribuzione normale e il decoder ricostruisce i dati a partire dalla distribuzione latente.

La loss function di un Variational Autoencoder combina due componenti principali. La prima è la perdita di ricostruzione, che misura quanto l'output ricostruito sia simile all'input originale e garantisce che la rete apprenda a ricostruire correttamente i dati. La seconda è un termine di regolarizzazione, basato sulla divergenza KL, che forza i codici latenti di dati simili a non discostarsi troppo tra loro e a seguire una distribuzione probabilistica predeterminata, tipicamente una Gaussiana standard. Insieme, questi due termini permettono al VAE di essere sia ricostruttivo che generativo.



(generazione di immagini di celebrità a partire da pattern estratte da diverse immagini)

Transformer



Le **reti neurali ricorrenti (RNN)** sono un tipo di rete neurale progettato per lavorare con sequenze di dati, dove l'ordine è importante.

Una tipologia di RNN nota è la **seq2seq(sequence-to-sequence)**, ossia una rete neurale che prende una **sequenza in input** e **produce una nuova sequenza** in output. Ad esempio, per la traduzione automatica si adopera la seq2seq.

Le seq2seq sono molto efficaci nella traduzione perché sanno comprendere la struttura della frase e ne generano una corretta nella lingua target. Per far ciò una rete neurale deve poter ricordare a lungo le informazioni nella sequenza, ed è per questo motivo che si utilizza il **modello LSTM** (long short-term memory, citata prima) che memorizza dipendenze tra termini in sequenze molto lunghe.

Tuttavia, all'aumentare della sequenza si ha instabilità, ed è per questo motivo che si deve abbandonare il concetto di ricorrenza. Si abbandona in favore ad un meccanismo che permetta di focalizzare l'attenzione su particolari elementi della sequenza e stabilire i riferimenti tra essi.

Questo meccanismo è anche adoperato a livello anatomico dal nostro cervello, dato che durante l'ascolto di un discorso ci si focalizza solo su alcune parole di maggiore importanza.

Le reti che fanno ciò vengono chiamati **transformer**.

Le RNN leggono le sequenze un passo alla volta, procedendo lentamente e con difficoltà su sequenze molto lunghe. I transformer, a differenza delle RNN, leggono la **sequenza in parallelo** garantendo:

- velocità maggiore;
- capacità di gestire sequenze molto lunghe;
- capacità di gestire parole lontane tra loro.

Con l'arrivo dei transformer sono nati i primi **Large Language Model**, ovvero sia un tipo di modello di intelligenza artificiale progettato per comprendere, generare e manipolare il linguaggio naturale.

L'obiettivo dei transformer è quello di trovare relazioni tra sequenze diverse, ad esempio:

- la **gatta** non attraversò la strada perché era troppo **stanca**;
- la gatta non attraversò la **strada** perché era troppo **larga**.

In questo caso la **self-attention** è un meccanismo fondamentale dei transformer, dato che permette di capire quali parole sono rilevanti o meno.

Il transformer è diviso in due parti principali:

- **Encoder** -> prende la sequenza di input e trasforma ogni parola in una rappresentazione numerica (**embedding**). Grazie alla self-attention e reti neurali feed-forward, l'encoder crea una **rappresentazione contestuale**, dove ogni parola "sa" com'è in relazione con altre parole della frase.
- **Decoder** -> a seconda dell'input passato dall'encoder il decoder si occuperà di **genera l'output**, generando un token alla volta e usando token precedenti per decidere il successivo.

La vera innovazione sta nella **self-attention**, che permette al modello di **pesare l'importanza di tutte le parole della frase rispetto a ciascuna parola**. La **multi-head attention** permette di calcolare più volte in parallelo la relazione tra le parole.

Riprendendo l'esempio di prima: "La gatta non attraversò la strada perché era stanca" -> "era" si riferisce a "gatta" dato che è anche presente "stanca".

Un'ulteriore innovazione nei transformer sono i **positional encoding**, che è un vettore aggiunto al contesto di ogni parola per indicare al modello la posizione della parola nella frase. Questo viene fatto perché le parole nei transformer vengono processate in parallelo e quindi non si sa l'ordine originale delle parole.

I transformer hanno bisogno di convertire il testo in numeri, per questo motivo si effettua una suddivisione del testo nella più piccola unità di comprensione, processo chiamato **tokenizzazione**. Inoltre, la rete non ha bisogno della rimozione delle **stopword** (punteggiatura, articoli, preposizioni), dello **stemming** (ossia ridurre le parole alla propria radice) (camminando -> cammina) e la **troncatura di parole troppo lunghe**, dato che questi modelli sono stati creati per comprendere le variazioni nelle forme delle parole (tempi verbali, errori di ortografia, ecc.).

La scelta dell'**embedder**, ossia la componente software che permette di convertire un testo in un vettore numerico è critica, dato che all'aumentare della qualità si ha un costo computazionale maggiore.

Una volta che si hanno i vettori rappresentativi del testo, si possono confrontare per vedere se sono simili tra loro attraverso la **similarità coseno**. Attraverso il coseno misuriamo l'angolo dei due vettori e l'interpretazione dei valori ci dà queste informazioni:

- **Vettori vicini** e nella stessa direzione → **similarità** ≈ 1 ;
- **Vettori perpendicolari** → **similarità** ≈ 0 ;
- **Vettori opposti** → **similarità** ≈ -1 ;

Abbiamo citato la **self-attention**, ma come si calcola? Per calcolare l'attenzione, il modello costruisce tre matrici a partire da X:

- **Query(Q)** -> matrice che rappresenta "cosa sto cercando" da ogni token. Si calcola così:

$$Q = XW_Q$$

dove:

- W_Q rappresenta la matrice dei pesi appresa per le query durante l'allenamento;
- Q è la matrice delle query e contiene le domande a cui si deve rispondere, ad esempio "a chi si riferisce era?"

- **Key(K)** -> matrice che rappresenta "quali token possono essere rilevanti per la query". Si calcola così:

$$K = XW_K$$

dove:

- W_K rappresenta la matrice dei pesi appresa per le parole chiavi durante l'allenamento;
- K è la matrice delle key, informazioni che ogni token può offrire per rispondere alla domanda, ad esempio "gatta", "strada" e "stanca";

- **Value(V)** -> matrice che rappresenta "informazioni effettive da prendere". Si calcola così:

$$V = XW_V$$

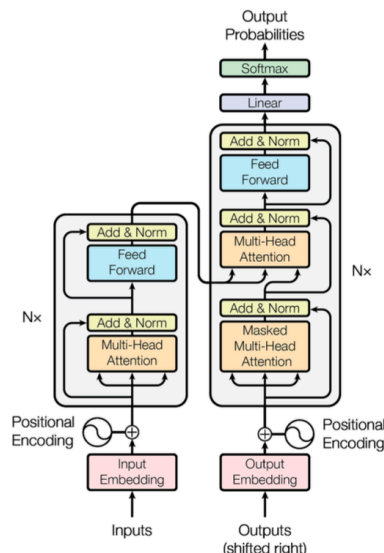
dove:

- W_V rappresenta la matrice dei pesi appresa per i valori;
- V rappresenta la matrice di valori che prenderò dai token rilevanti.

La prima operazione nel calcolo d'attenzione è il prodotto scalare tra Q e K. Si fa attraverso le matrici citate prima e la formula è:

$$\text{Attention scores} = QK^T$$

Ogni elemento (i,j) indica quanto il **token i dovrebbe prestare attenzione al token j**. In pratica si misura la similarità tra query e key.



Dopo il prodotto si riscaldano i valori, per evitare valori troppo piccoli o grandi del gradiente. L'applicazione della maschera è conseguenza dei passaggi citati prima, dato che si può lavorare con sequenze di lunghezza variabile. La maschera applicata può essere di due tipi:

- **Padding mask:** vengono riempite con token di padding che non influenzano il calcolo dell'attenzione;
- **Future attention mask:** utilizzata durante la fase di generazione del testo per bloccare token futuri, in modo che non si bari durante la generazione(data leakage).

La **softmax trasforma i punteggi di attenzione in probabilità**, pesi tra 0 e 1, la cui somma di ogni riga fa 1. Quindi trasforma i punteggi grezzi in valori interpretabili dal modello.

L'embedding originale del token viene aggiornato, dato che si combinano le informazioni dagli altri token pesate in base alla loro rilevanza. Il nuovo embedding tiene conto non solo della parola, ma del contesto in cui si trova.

LLM

I modelli di intelligenza solitamente sono derivati dai Transformer e si utilizzano per comprendere e generare linguaggio umano, codice e molto altro. Essi vengono addestrati su una grandissima quantità di dati testuali, catturando così la complessità e le sfumature del linguaggio umano.

Il successo dei transformer è legato a:

- **meccanismi di multi-head attention;**
- **transfer learning** -> il modello viene allenato su grandi quantità di dati per un compito generale, in questo modo apprende il linguaggio (BERT è stato pre addestrato su wikipedia e Book corpus)-> lo stesso modello poi viene riaddestrato un po', con pochi dati per un compito più specifico(**fine tuning**);
- **scaling up delle reti** -> l'aumento del numero di parametri, layer o dimensione di embedding permette di imparare strutture più complesse(però si necessita di più calcolo);
- **parallelizzazione** -> si possono utilizzare centinaia di GPU in parallelo per l'elaborazione token-per-token -> permesso di scalare su dataset molto grandi e ha permesso di creare modelli con ampio numero di parametri.

Gli LLM solitamente svolgono due compiti fondamentali:

- 1) **Generativi** -> addestrati per prevedere il token successivo in una frase, a partire dai precedenti. Questi modelli sfruttano soltanto il **decoder del transformer** usando una maschera **Future attention mask**. Sono ideali per generazione di free-text. -> CHAT GPT
- 2) **Auto-encoding** -> addestrati per classificare token di una frase. Questi modelli utilizzano solo l'encoder e hanno accesso all'intero input senza maschera. Controlla sia prima del token analizzato e sia dopo. Ad esempio, "Quando ero a scuola, il banco si è rotto" banco può assumere sia valore di scrivania sia di banca, per questo motivo si controlla prima del token e dopo. -> BERT

Nelle AI generative, la temperatura è un parametro che il motore di generazione del modello usa quando deve scegliere il prossimo token tra tanti candidati possibili.

Il modello ha una distribuzione di probabilità, ovvero:

- Token molto probabile -> alta probabilità;
- Token raro -> bassa probabilità;

La temperatura modifica questa distribuzione, per renderla più rigida o più morbida.

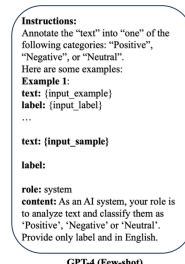
- Se la temperatura è bassa (0.0-0.3) vince sempre il token più probabile -> la risposta è più deterministica, rigida e coerente.
- Se la temperatura è alta(0.7-1.2) anche token meno probabili diventano possibili -> risposta più creativa e sorprendente.

Le 4 fasi del transfer Learning prevedono:

- 1) Scegliere il modello pre-addestrato che ha già imparato la lingua(grammatica, sintassi, significato delle parole, ragionamento, ecc.) -> GPT(generazione) o BERT(classificazione);
- 2) Prendere un piccolo dataset specifico per il compito finale -> qualche migliaio di esempi;
- 3) Calcolo della loss e del gradiente a ogni epoca -> modello fa previsioni sbaglia e si calcola errore e gradiente;
- 4) Si aggiornano i pesi attraverso l'algoritmo di backpropagation.

Il prompt è l'input che diamo ad un modello di linguaggio per fagli svolgere un compito ed deve essere chiaro e preciso. Quando si interagisce in una chat il modello riceve tre tipi di informazioni:

- Prompt di sistema -> istruzioni generale che dice al modello come comportarsi nella conversazione -> impostare una personalità, regole generali o topic fissato;
- Prompt utente -> quello che richiede l'utente;
- Prompt assistente -> risposte del modello stesso, che costruiscono il contesto.



```
Instructions:
Annotate the "text" into "one" of the
following categories: "Positive",
"Negative", or "Neutral".
Here are some examples:
Example 1:
text: {input_example}
label: {input_label}
...
text: {input_sample}
label:

role: system
content: As an AI system, your role is
to analyze text and classify them as
'Positive', 'Negative' or 'Neutral'.
Provide only label and in English.
```

GPT-4 (Few-shot)

I modelli possono anche migliorare le proprie capacità, non basandosi solo sulla conoscenza interna. Per questo motivo si è sviluppata una metodologia chiamata RAG(Retrieval Augmented Generation), che consente di guidare gli LLM con informazioni aggiornate o dettagliate estratte da una base di conoscenza.

Un esempio è la nuova versione di chat GPT che genera il testo a partire dalle proprie informazioni e da ricerche svolte sulla rete.

Quando si lavora con grandi testi, come libri o articoli, è inefficiente fare l'embedding dell'intero documento. Si suddivide quindi il testo in **chunk** di dimensione limitata, spesso con **overlapping**(alcune parole alla fine di un chunk vengono ripetute nel chunk successivo, per mantenere la continuità del testo) per mantenere il contesto. Gli embedding di questi chunk vengono memorizzati in un **vector database**, che permette di recuperare rapidamente i segmenti più simili(similarità coseno) a una query dell'utente basata sul significato semantico, facilitando ricerche efficienti e contestuali.